Ben-Gurion University of the Negev
Faculty of Engineering Sciences
Department of Mechanical Engineering

Final Project

Graduate Course in Mechatronic Systems

Nonlinear Control and Real-Time Digital Implementation

Adaptive RBF Neural-Network Tracking Control and
Off-Policy Adaptive Dynamic Programming,
with Real-Time Implementation on the Raspberry Pi Pico

Roee Zehavi 209146216 roeeze@post.bgu.ac.il

Etay Baron 209438910 etaybaro@post.bgu.ac.il

Instructor: Prof. Shai Arogeti

Code, data and this report:
https://github.com/ThEpiCake/Mechatronics_Final_Project

July 16, 2026

Contents

1 Introduction and Project Overview

This project develops and compares two nonlinear controllers for a single mechatronic process and implements both of them in real time on a Raspberry Pi Pico. The process is a mass on a hardening (Duffing) spring with linear viscous damping, driven through a first-order force actuator; it is the third-order (*jerk*) nonlinear plant introduced in Homework 1, reused here in accordance with the project guidelines. The two controllers are:

- **Branch A** — an adaptive Radial-Basis-Function neural-network (RBF-NN) controller that makes the output track a smooth reference trajectory while learning the unknown drift online, with a Lyapunov-based weight-adaptation law;
- **Branch B** — a nonlinear optimal regulator obtained from data by *off-policy* adaptive dynamic programming (ADP), i.e. policy iteration on a parameterized value function without knowledge of the drift.

Both controllers use the same control-affine model $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})u$, in which the input field $\boldsymbol{g}$ is known and the drift $\boldsymbol{f}$ is treated as unknown. After the two designs are validated in Python, they are ported to MicroPython and run on the Pico's two cores: the secondary core integrates the nonlinear process in real time while the main core executes the active controller, exchanging the state vector and the control input every sampling period.

To summarize the outcome up front: the adaptive RBF-NN controller, starting from zero knowledge of the drift, tracks the reference with a steady-state RMS error of $\approx 2\%$ of the amplitude, and its discrete emulation matches the continuous design down to a 20 Hz sampling rate; the data-driven ADP regulator reproduces the LQR solution near the origin to 1.1% and cuts the accumulated cost by 41–56% relative to the (stable) uncontrolled process; and both controllers run *on the physical Pico* under a verified two-core real-time budget, with the on-chip responses matching the computer results to a few millimetres. Every quantitative claim in this report is reproduced by the accompanying code and, for Section 5, measured on the board itself.

The report follows the structure mandated by the assignment: Section 2 presents the nonlinear process, its control-affine form and the Lyapunov analysis of the zero-input system (work items 1.1–1.7); Section 3 develops Branch A (items A1–A5); Section 4 develops Branch B (items B1–B8); Section 5 describes the real-time two-core Pico implementation and the execution-time measurements (items C1–C5); and Section 6 summarizes the results and their limitations.

2 Nonlinear Process Model and Analysis

2.1 Physical process, model source, and modelling assumptions

The selected process is a single mechanical degree of freedom driven through a first-order force actuator. A mass m slides against linear viscous damping c_1 and is restrained by a *hardening (Duffing) spring* whose restoring force $k_1 y + k_3 y^3$ combines a linear and a cubic term. The mass is pushed by the force F produced by an actuator (a voice-coil / servo valve) whose output lags its command u with a first-order time constant τ. The governing equations are

$$\begin{aligned} m\,\ddot{y} + c_1\dot{y} + k_1 y + k_3 y^3 &= F, \\ \tau\dot{F} + F &= k_a u. \end{aligned} \tag{1}$$

The cubic term is the classical Duffing nonlinearity [1, 2]; the first-order actuator is a standard servo model. This is the same physical process analysed in Homework 1, reused here as permitted by the assignment; the structure and the mass/stiffness/gain are unchanged, while three parameters are re-tuned for this project (actuator lag τ : $0.05 \to 0.2$ s, cubic stiffness

$k_3 : 5 \to 40$, damping $c_1 : 4 \to 1$) so that the process is strongly nonlinear at a moderate amplitude and lightly damped. The physical parameters (Table 1) are fixed for simulation, but — as required — the controllers of Sections 3 and 4 do *not* use them: only the input gain is assumed known, while the drift is treated as an unknown smooth function.

Table 1: Physical parameters and derived model coefficients.

Symbol	Value	Meaning	Symbol	Value	Meaning
m	1.0	mass [kg]	$a_1 = c_1/m + 1/\tau$	6	(Eq. 2)
c_1	1.0	damping [N s/m]	$a_2 = k_1/m + c_1/(\tau m)$	25	
k_1	20.0	linear stiffness [N/m]	$a_3 = k_1/(\tau m)$	100	
k_3	40.0	cubic stiffness [N/m^3]	$c = k_a/(\tau m)$	5	input gain
τ	0.2	actuator lag [s]	$b_1 = k_3/(\tau m)$	200	coeff. of y^3
k_a	1.0	actuator gain [N/V]	$b_2 = 3k_3/m$	120	coeff. of $y^2\dot{y}$

Assumptions. (i) the full state is measured (spec 1.7); (ii) the actuator is an ideal first-order lag; (iii) the parameters are constant; (iv) the input is limited to $|u| \le u_{\max} = 40$ V; (v) the operating range is the box in Table 2.

2.2 Mathematical model, operating range, known $g(x)$ and unknown $f(x)$

Choosing the companion state $\boldsymbol{x} = [x_1, x_2, x_3]^\top = [y, \dot{y}, \ddot{y}]^\top$, i.e. position, velocity and acceleration, and eliminating F between the two equations of (1) (solve the mechanical equation for F, differentiate once using $\frac{d}{dt}y^3 = 3y^2\dot{y}$, and substitute into the actuator equation) yields the exact scalar third-order *jerk* model

$$\dddot{y} + a_1\ddot{y} + a_2\dot{y} + a_3 y + b_1 y^3 + b_2 y^2\dot{y} = c\,u. \tag{2}$$

In state-space this is the control-affine form required by the assignment,

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})\,u, \qquad \boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} x_2 \\ x_3 \\ -D(\boldsymbol{x}) \end{bmatrix}, \quad \boldsymbol{g}(\boldsymbol{x}) = \begin{bmatrix} 0 \\ 0 \\ c \end{bmatrix}, \tag{3}$$

with the scalar drift in the input channel

$$D(\boldsymbol{x}) = a_1 x_3 + a_2 x_2 + a_3 x_1 + b_1 x_1^3 + b_2 x_1^2 x_2. \tag{4}$$

The input field $\boldsymbol{g}$ is *constant*, hence known to the controller; the drift D (through $\boldsymbol{f}$) is treated as the *unknown* smooth nonlinear function. The system has relative degree three with respect to the output $y = x_1$, and $\boldsymbol{f}(\boldsymbol{0}) = \boldsymbol{0}$, so the regulation equilibrium is already the origin (spec 1.5). The operating range and input limit are listed in Table 2; they enclose the tracking tube of Section 3 with margin.

Table 2: Operating range (companion state) and input limit.

	y [m]	$\dot{y}$ [m/s]	$\ddot{y}$ [m/s^2]	F [N]	u [V]
range	$[-0.8, 0.8]$	$[-2.0, 2.0]$	$[-11, 11]$	$\lesssim 50$	$[-40, 40]$

2.3 Regulation equilibrium and Lyapunov stability of the zero-input system

Setting $\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{0}$ gives $x_2 = x_3 = 0$ and $a_3 x_1 + b_1 x_1^3 = x_1(a_3 + b_1 x_1^2) = 0$; since $a_3, b_1 > 0$ the only real solution is $x_1 = 0$, so the **origin is the unique equilibrium** of the zero-input system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$.

The stability analysis is cleanest in the physically equivalent coordinates $\boldsymbol{z} = [y, \dot{y}, F]^\top$, related to $\boldsymbol{x}$ by the global diffeomorphism $F = mx_3 + c_1x_2 + k_1x_1 + k_3x_1^3$. In these coordinates the zero-input dynamics are

$$\dot{z}_1 = z_2, \qquad \dot{z}_2 = \tfrac{1}{m}\Big(z_3 - c_1z_2 - k_1z_1 - k_3z_1^3\Big), \qquad \dot{z}_3 = -\tfrac{1}{\tau}z_3. \tag{5}$$

Consider the energy-based Lyapunov candidate (kinetic + spring potential + actuator storage)

$$V(\boldsymbol{z}) = \tfrac{1}{2}mz_2^2 + \tfrac{1}{2}k_1z_1^2 + \tfrac{1}{4}k_3z_1^4 + \tfrac{1}{2}\gamma z_3^2, \qquad \gamma > 0, \tag{6}$$

which is positive definite and radially unbounded because $k_1, k_3 > 0$. Differentiating along (5), the spring and inertia terms cancel and

$$\dot{V} = -c_1z_2^2 - \frac{\gamma}{\tau}z_3^2 + z_2z_3 = -\begin{bmatrix} z_2 \\ z_3 \end{bmatrix}^\top \begin{bmatrix} c_1 & -\frac{1}{2} \\ -\frac{1}{2} & \gamma/\tau \end{bmatrix} \begin{bmatrix} z_2 \\ z_3 \end{bmatrix}. \tag{7}$$

The 2×2 matrix is positive definite iff $c_1 > 0$ and $c_1\gamma/\tau - \frac{1}{4} > 0$, i.e. whenever $\gamma > \tau/(4c_1)$. Choosing $\gamma = \tau/c_1$ satisfies this, so $\dot{V} \le 0$ everywhere, with $\dot{V} = 0$ only on the set $\{z_2 = z_3 = 0\}$. On that set $\dot{z}_2 = 0$ forces $k_1z_1 + k_3z_1^3 = 0$, i.e. $z_1 = 0$; hence the largest invariant set contained in $\{\dot{V} = 0\}$ is the origin. By LaSalle's invariance principle the origin is asymptotically stable, and since V is radially unbounded and the coordinate map is a global diffeomorphism, the result is in fact *global* (and therefore local, as required); analytically, $\dot{V}$ is a negative-semidefinite quadratic form in $(\dot{y}, F)$ for the chosen γ. Figure 1 confirms the conclusion numerically: trajectories from a fan of initial conditions spiral into the origin, and $V(t)$ decreases monotonically with $\dot{V} \le 0$ throughout.

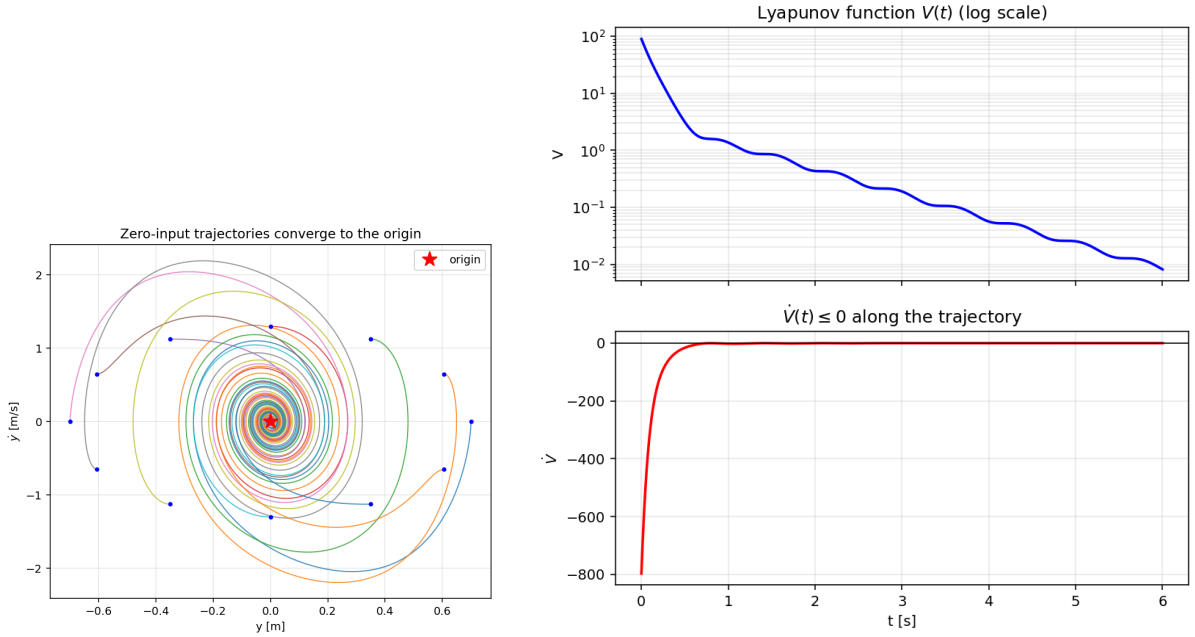

Figure 1: Zero-input system. Left: phase-plane convergence to the origin. Right: the Lyapunov function decreases with $\dot{V} \le 0$ along a trajectory.

2.4 Significance of the nonlinear dynamics

Linearizing (3) about the origin (dropping $b_1x_1^3$ and $b_2x_1^2x_2$) gives $\dot{\boldsymbol{x}} = A\boldsymbol{x} + Bu$ with

$$A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a_3 & -a_2 & -a_1 \end{bmatrix}, \tag{8}$$

whose eigenvalues are -5 and $-0.5 \pm 4.44j$ (open-loop stable, lightly damped). The nonlinearity is significant over the operating range: at $y = 0.7$ m the cubic force $k_3 y^3$ is 98% of the linear force $k_1 y$. Comparing the zero-input free response of the nonlinear model with its linearization (Fig. 2), the two are indistinguishable at small amplitude ($y_0 = 0.15$ m, peak deviation 2.5 mm) but differ markedly at large amplitude ($y_0 = 0.7$ m, peak deviation 0.16 m): the hardening spring stiffens the effective restoring force, so the nonlinear response is faster and phase-advanced. The responsible terms are the cubic stiffness $b_1 y^3$ and the nonlinear damping cross-term $b_2 y^2 \dot{y}$; both vanish to first order at the origin, which is why the linearization is only locally valid.

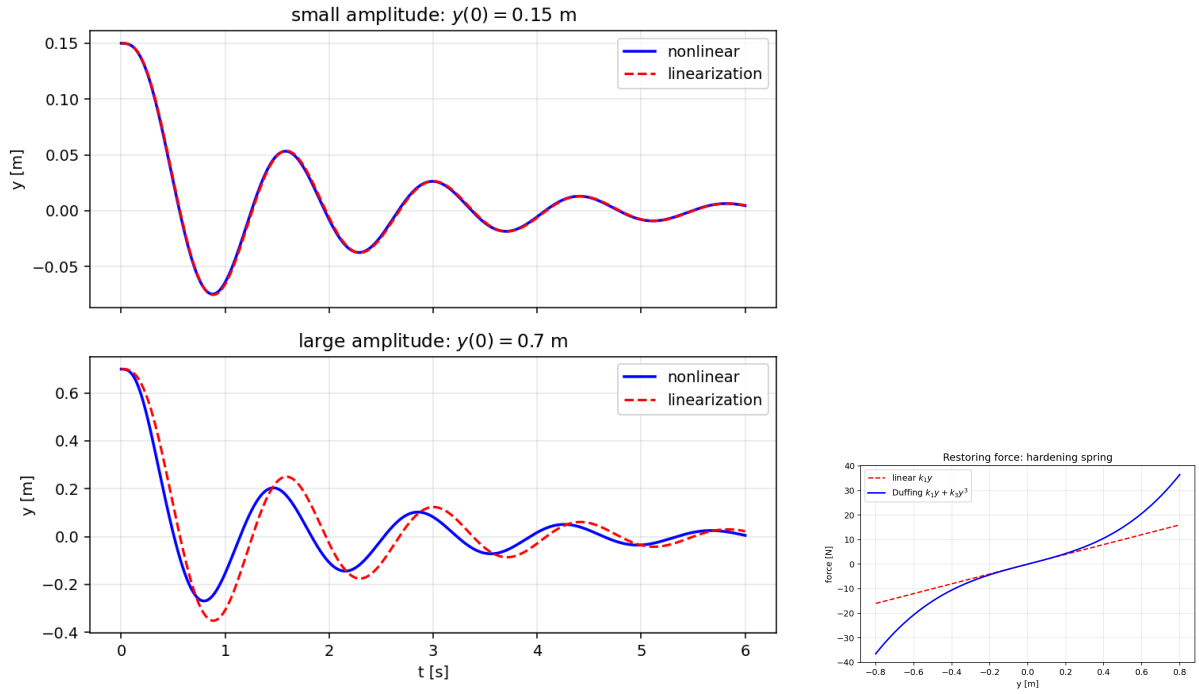

Figure 2: Nonlinear vs. linearized model. Left: free response at small and large amplitude. Right: the hardening restoring force $k_1 y + k_3 y^3$.

3 Adaptive RBF-NN Tracking Controller

3.1 Tracking objective and nominal continuous-time controller

The output $y = x_1$ must track the smooth reference $y_d(t) = A \sin(\omega t)$ with $A = 0.5$ m and $\omega = 2$ rad/s; its derivatives $\dot{y}_d, \ddot{y}_d, \dddot{y}_d$ are available in closed form. Defining the tracking error $e = y - y_d$ and the error vector $\boldsymbol{\xi} = [e, \dot{e}, \ddot{e}]^\top$, and recalling that the process has relative degree three with $\dddot{y} = c\,u - D(\boldsymbol{x})$ (Eqs. 2–4), the *nominal* feedback-linearizing controller (drift D assumed known) is

$$u = \frac{1}{c}\Big[\, D(\boldsymbol{x}) + \dddot{y}_d - \lambda_2 \ddot{e} - \lambda_1 \dot{e} - \lambda_0 e \,\Big], \tag{9}$$

which cancels the drift and imposes the linear error dynamics $\dddot{e} + \lambda_2 \ddot{e} + \lambda_1 \dot{e} + \lambda_0 e = 0$, i.e. $\dot{\boldsymbol{\xi}} = \Lambda \boldsymbol{\xi}$ with Λ the companion matrix of the gains $(\lambda_2, \lambda_1, \lambda_0) = (21, 146, 336)$, whose roots are the chosen error poles $\{-6, -7, -8\}$. This yields exact asymptotic tracking (Fig. 3, dotted).

3.2 RBF approximation, adaptive controller, weight-adaptation law, and stability

When D is unknown it is approximated by a *normalized* Gaussian RBF network,

$$D(\boldsymbol{x}) = W^{*\top}\boldsymbol{\phi}(\boldsymbol{x}) + \varepsilon(\boldsymbol{x}), \qquad \phi_i(\boldsymbol{x}) = \frac{r_i(\boldsymbol{x})}{\sum_j r_j(\boldsymbol{x})}, \quad r_i(\boldsymbol{x}) = \exp\Big(-\frac{\|(\boldsymbol{x}-\boldsymbol{c}_i)\oslash \boldsymbol{h}\|^2}{2\eta^2}\Big), \tag{10}$$

where $\boldsymbol{c}_i$ are $6\times 6\times 6 = 216$ centers on a regular grid, $\boldsymbol{h}$ is the per-axis half-width used to normalize the coordinates (so a single width handles the three different state scales), $\eta = 0.24$ is the normalized width (≈ 0.6 of the grid spacing), $\oslash$ is elementwise division, and ε is the bounded approximation error ($|\varepsilon| \le \bar{\varepsilon}$ on the covered box). The grid deliberately spans the *tracking box* $[-0.7, 0.7]\times[-1.3, 1.3]\times[-3, 3]$ — the tube visited in *steady-state* tracking (amplitudes 0.5, 1.0, 2.0 per axis, plus margin) — rather than the full operating box of Table 2: concentrating the 216 centers where the closed-loop trajectory lives improves the conditioning and the local resolution of the approximation, and the drift needs to be learned only along that tube. (The cold-start transient briefly exceeds the third axis, $|\ddot{y}|$ reaching ≈ 4.3; there the saturated input, the projection bound and the zero-input stability of Section 2.3 keep the loop bounded, and the UUB analysis below applies once the state re-enters the box.) An offline least-squares fit of the network to D over the tracking box quantifies the approximation error: $\max|\varepsilon| \approx 14$ m/s^3 and RMS ≈ 2.0 m/s^3, i.e. 5.6% and 0.8% of $\max|D| \approx 255$ m/s^3 — this is the $\bar{\varepsilon}$ entering the bound (15) (which is, as usual, conservative relative to the observed steady error). The *normalized* (partition-of-unity) form is essential: a plain Gaussian basis is nearly linearly dependent here and would require astronomically large weights, whereas the normalized network has well-conditioned, local weights $\approx$ the local values of the drift. The estimate $\widehat{D}(\boldsymbol{x}) = \widehat{W}^{\top}\boldsymbol{\phi}(\boldsymbol{x})$ replaces D in (9),

$$u = \frac{1}{c}\Big[\,\widehat{W}^{\top}\boldsymbol{\phi}(\boldsymbol{x}) + \dddot{y}_d - \lambda_2\ddot{e} - \lambda_1\dot{e} - \lambda_0 e\,\Big], \tag{11}$$

so that, with $\widetilde{W} = \widehat{W} - W^*$ and $\boldsymbol{b} = [0, 0, 1]^{\top}$, the error dynamics become

$$\dot{\boldsymbol{\xi}} = \Lambda\boldsymbol{\xi} + \boldsymbol{b}(\widetilde{W}^{\top}\boldsymbol{\phi} - \varepsilon). \tag{12}$$

Standing assumptions. The stability argument below uses: (i) full-state measurement and a known constant input gain c (Section 2.1), so the error vector $\boldsymbol{\xi}$ and the features $\boldsymbol{\phi}(\boldsymbol{x})$ are computable online; (ii) the existence of an ideal weight vector W^* with $\|W^*\|_\infty \le W_{\max}$ such that $|\varepsilon(\boldsymbol{x})| \le \bar{\varepsilon}$ on the tracking box — guaranteed by the universal-approximation property of Gaussian RBF networks for the smooth drift D on a compact set [3], and quantified for this particular network by the offline fit above.

Lyapunov design. Let $P = P^{\top} \succ 0$ solve $\Lambda^{\top}P + P\Lambda = -Q$ with $Q = \mathrm{diag}(50, 10, 1) \succ 0$, and take

$$V = \boldsymbol{\xi}^{\top}P\,\boldsymbol{\xi} + \frac{1}{\gamma}\widetilde{W}^{\top}\widetilde{W}. \tag{13}$$

Along (12), choosing the weight-adaptation law as the Lyapunov gradient with a projection that keeps $\|\widehat{W}\|_\infty \le W_{\max}$,

$$\dot{\widehat{W}} = \mathrm{Proj}\Big\{-\gamma\,(\boldsymbol{b}^{\top}P\,\boldsymbol{\xi})\,\boldsymbol{\phi}(\boldsymbol{x})\Big\}, \qquad \gamma = 4\times 10^4, \;\; W_{\max} = 600, \tag{14}$$

the indefinite cross term cancels and

$$\dot{V} \le -\boldsymbol{\xi}^{\top}Q\,\boldsymbol{\xi} - 2(\boldsymbol{b}^{\top}P\boldsymbol{\xi})\varepsilon \le -\lambda_{\min}(Q)\|\boldsymbol{\xi}\|^2 + 2\|P\boldsymbol{b}\|\,\bar{\varepsilon}\,\|\boldsymbol{\xi}\|. \tag{15}$$

The projection in (14) preserves this inequality: it modifies the update only when a weight sits at its bound ($|\widehat{w}_i| = W_{\max}$) and the gradient drive pushes it further out, and it then

zeroes exactly that component; since $\|W^*\|_\infty \le W_{\max}$, the resulting correction term has a non-positive inner product with $\widetilde{W}$, so it adds a non-positive contribution to $\dot{V}$ and (15) still holds (this is the boundary-stopping projection implemented in `weight_dot` of `rbf_adaptive.py`, Appendix A). Hence $\dot{V} < 0$ whenever $\|\boldsymbol{\xi}\| > 2\|P\boldsymbol{b}\|\bar{\varepsilon}/\lambda_{\min}(Q)$: the tracking error and the weights are *uniformly ultimately bounded* [2], and $\boldsymbol{\xi}$ converges to a residual set whose size is set by the RBF approximation error $\bar{\varepsilon}$ (projection guarantees $\widehat{W}$ stays bounded without biasing it toward zero, unlike σ-modification).

Choice of the design parameters. The error poles $\{-6, -7, -8\}$ place the error bandwidth well above the reference frequency (2 rad/s) while keeping the nominal input within the $\pm u_{\max}$ limit. $Q = \mathrm{diag}(50, 10, 1)$ weights the position error most, which makes the adaptation row $\boldsymbol{b}^\top P = [0.074, 0.112, 0.029]$ respond primarily to e and $\dot{e}$ rather than to the noisy-looking $\ddot{e}$. The adaptation gain $\gamma = 4 \times 10^4$ compensates the small magnitude of $\boldsymbol{b}^\top P\boldsymbol{\xi}$; it was chosen so the drift is learned within a few reference periods, and the sampling sweep of Section 3.4 confirms that this fast adaptation still discretizes stably far below the design rate. The width $\eta = 0.24$ (≈ 0.6 of the grid spacing) gives neighbouring Gaussians enough overlap for a smooth partition of unity without smearing the local resolution, and $W_{\max} = 600$ sits a factor ≈ 3 above the largest transient weight magnitude ($\max_i |\widehat{w}_i| \approx 206$ during adaptation; the converged weights settle below ≈ 150), so the projection guards transients without constraining the converged network.

Figure 3 shows the adaptive controller, starting from $\widehat{W}(0) = \mathbf{0}$, driving the tracking error down to a steady RMS of 1.0×10^{-2} m ($\approx 2\%$ of the amplitude; steady RMS here denotes the RMS of e over the second half of the 30 s run); the corresponding velocity and acceleration states are shown in Fig. 4, and the control input (Fig. 5, left) remains well inside the ± 40 V actuator limit throughout, with brief adaptation transients early in the run. Meanwhile the network output $\widehat{D}$ learns the true drift online (Fig. 6), the weight norm remains bounded ($\|\widehat{W}\| \to 367$), and representative individual weights (Fig. 5, right) converge to steady values that encode the drift locally.

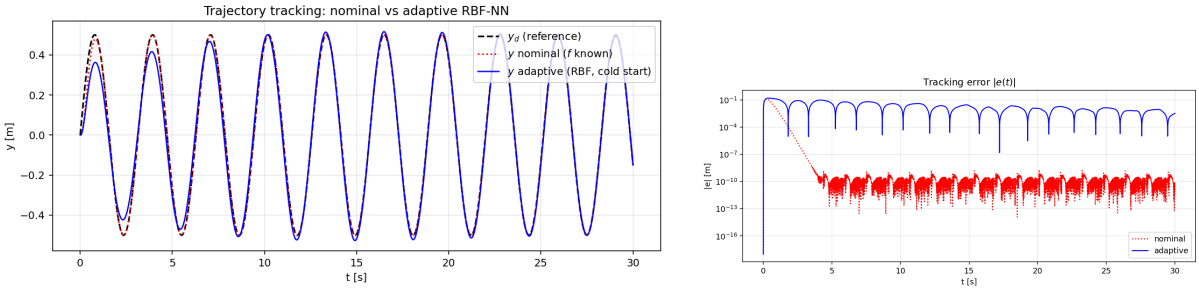

Figure 3: Tracking with the nominal (f known) and adaptive RBF-NN controllers. The adaptive error decreases as the network learns; the nominal error sits at the integration-tolerance floor.

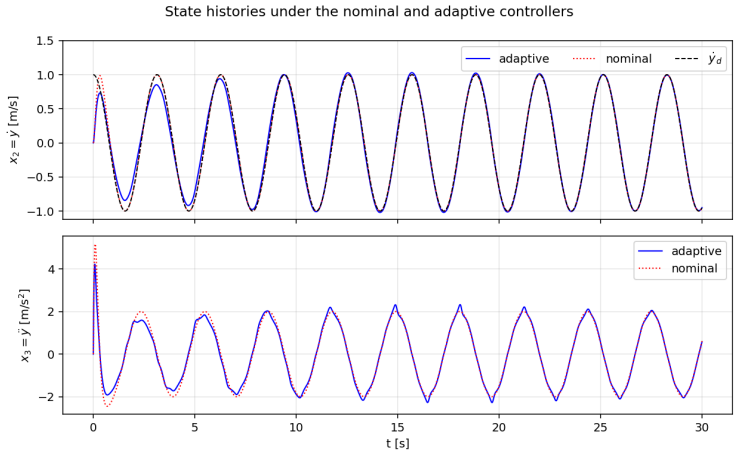

Figure 4: State histories $x_2 = \dot{y}$ and $x_3 = \ddot{y}$ under the nominal and adaptive controllers (the output $x_1 = y$ is shown in Fig. 3).

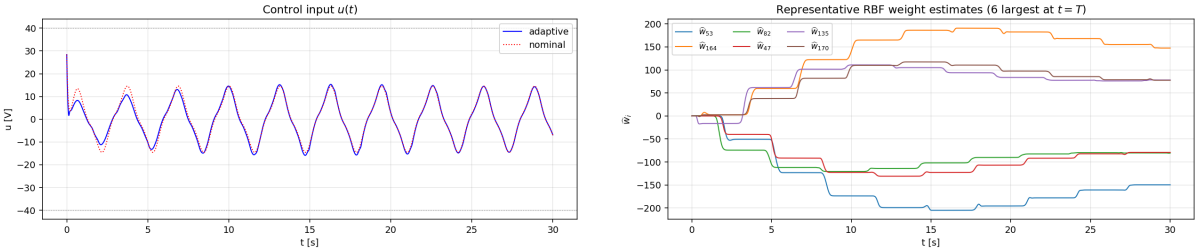

Figure 5: Left: control input of the nominal and adaptive controllers, within the ±40 V limit. Right: representative RBF weight estimates (the six largest at $t = T$), converging to bounded steady values.

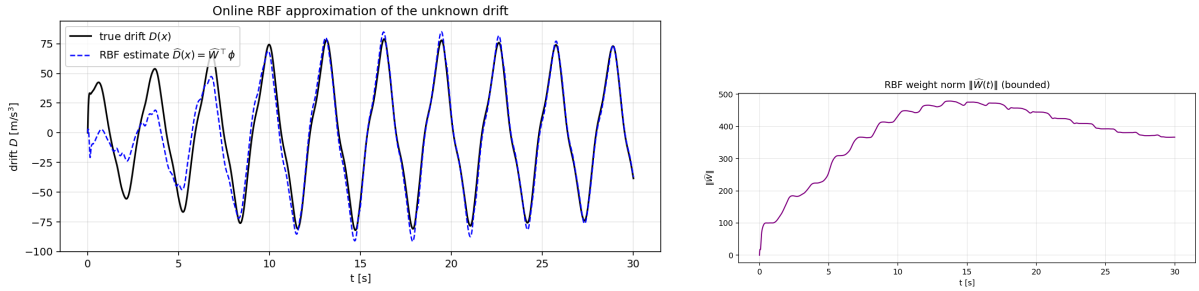

Figure 6: Online RBF approximation of the unknown drift $D(\boldsymbol{x})$ (left) and the bounded weight norm (right).

3.3 Discrete-time realization

The continuous design is implemented digitally by the course's *emulation* (indirect digital-design) approach [7]: the controller runs at a sampling period T_s, the input is held constant between samples (ZOH), and the plant evolves in continuous time (integrated exactly by RK4 sub-stepping). The per-sample calculation order is: (1) read the full state $\boldsymbol{x}_k$; (2) form the reference, errors $\boldsymbol{\xi}_k$ and features $\boldsymbol{\phi}_k$; (3) compute u_k from (11) and saturate to $|u| \le u_{\max}$; (4) hold u_k and advance the plant one period; (5) update the weights by *forward Euler*, $\widehat{W}_{k+1} = \widehat{W}_k + T_s\,\dot{\widehat{W}}_k$ with $\dot{\widehat{W}}_k$ from (14). The control input is limited to $\pm u_{\max} = 40$ V and the weights by the same projection bound $W_{\max}$.

3.4 Computer-based validation and continuous/discrete comparison

All comparisons use the same process, reference and initial condition $\boldsymbol{x}(0) = \mathbf{0}$. The adaptive controller is first compared with the nominal one (Fig. 3). The continuous design is then compared with its discrete realization over a sweep of eleven sampling rates from 1 kHz down to 10 Hz; three representative responses are shown in Fig. 7 and the steady RMS error of the whole sweep in Fig. 8 (in this comparison the steady RMS is computed over the final 30% of each run, hence the slightly smaller values than the half-run figure quoted in Section 3.2). At fine rates ($f_s \geq 100$ Hz) the discrete response is indistinguishable from the continuous one (steady RMS 8.9–9.1 $\times 10^{-3}$ m); the error grows only mildly down to $f_s = 20$ Hz ($T_s = 50$ ms, RMS 1.03×10^{-2} m); at $f_s = 15$ Hz ($T_s \approx 67$ ms) the loop *clearly degrades*.

The degradation mechanism is the sampled adaptation law, not the control law itself. The weight update (14) is a fast local subsystem whose rate is set by $\gamma \, \|\boldsymbol{\phi}\|^2 \, \|\boldsymbol{b}^\top P\|$; forward-Euler integration of that subsystem is stable only while T_s is small against its local time constant. Early in the run the weights are small and every rate converges, but once $\|\widehat{W}\|$ has grown the effective loop gain of the error-adaptation interconnection increases, the 15 Hz Euler update starts to overshoot, the weights oscillate against the projection bound, and under the resulting saturated, erratic input the output collapses to a small-amplitude limit cycle far from the reference (Fig. 7, right) — the *tracking error* becomes large while the response itself stays bounded. This is the classical emulation-design failure: the digital controller inherits the continuous design only while the sampling is fast relative to *every* closed-loop mode, including the adaptation dynamics.

Based on this sweep, $T_s = 5$ ms ($f_s = 200$ Hz) is adopted as the *initial design* sampling period for the digital implementation: it is the finest rate that leaves generous real-time headroom on an embedded target while sitting a factor ≈ 13 above the observed 15 Hz degradation point. The measured execution times on the Raspberry Pi Pico revisit this choice in Section 5.

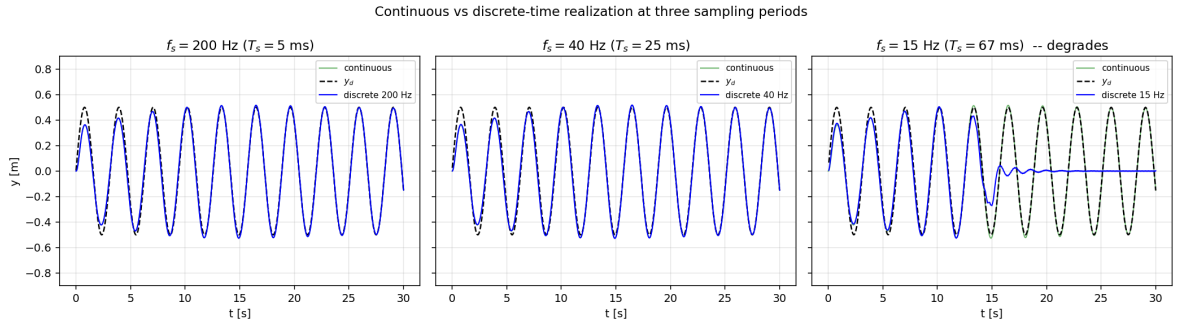

Figure 7: Continuous vs. discrete-time realization at three sampling periods. Fine and intermediate rates match the continuous design; the coarse rate (15 Hz) degrades.

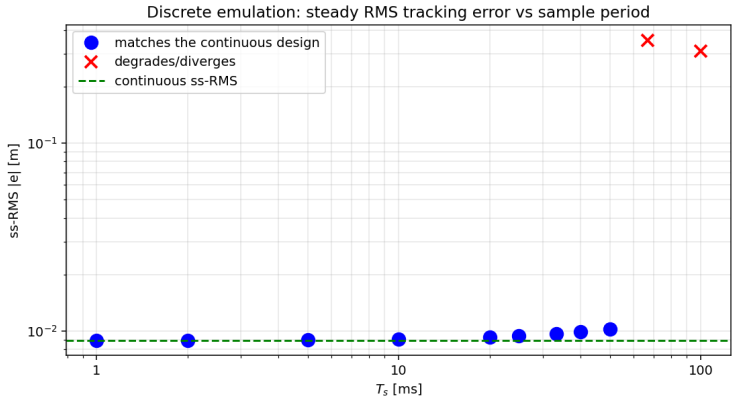

Figure 8: Sampling sweep of the discrete emulation: steady RMS tracking error vs. the sampling period T_s. Blue markers match the continuous steady RMS (dashed); red crosses mark the degraded/diverging rates ($f_s \leq 15$ Hz).

4 Nonlinear Off-Policy ADP

4.1 Regulation problem, performance index, and initial policy

Branch B seeks the optimal state-feedback policy that regulates the same process to the origin while minimizing the infinite-horizon quadratic cost

$$J(\boldsymbol{x}_0) = \int_0^\infty \left(\boldsymbol{x}^\top Q\,\boldsymbol{x} + R\,u^2\right) dt, \qquad Q = \mathrm{diag}(300, 20, 1) \succ 0, \quad R = 0.1 > 0. \tag{16}$$

Q penalizes the position most heavily (regulation of y is the objective) and the velocity and acceleration progressively less; the small R makes control relatively cheap, so the optimal policy is allowed to act aggressively. The *initial policy* is $u_0(\boldsymbol{x}) = 0$; it is *admissible* because the zero-input system $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x})$ is globally asymptotically stable (Section 2.3), so it stabilizes the process on the operating region and yields a finite cost J (indeed $\int_0^\infty \boldsymbol{x}^\top Q\boldsymbol{x}\,dt < \infty$). This ties the admissibility of u_0 directly to the Part 1 Lyapunov result.

4.2 Data generation

The drift f is unknown, so the value function is learned from measured data using an *off-policy* scheme [5, 4]. A single trajectory is generated by applying a bounded, sufficiently exciting *behavior* input — the sum of five non-harmonically related sinusoids, $u(t) = \frac{A_e}{5}\sum_{k=1}^{5}\sin(\omega_k t)$ with $A_e = 8$ V and $\{\omega_k\} = \{1.3, 3.7, 7.1, 11.9, 17.0\}$ rad/s — to the true plant from $\boldsymbol{x}_0 = [0.6, 0, 0]^\top$. States and input are sampled at $dt = 1$ ms for 20 s (= 400 integration windows of $T = 50$ ms). The state remains within the declared operating range throughout (peak $|\boldsymbol{x}| = [0.6, 1.43, 5.95]$ versus the box $[0.8, 2, 11]$) and $|u| \leq 7.7$ V $< u_{\max}$ (Fig. 9). The same data set is reused for every policy-iteration step.

4.3 Value-function approximation and policy improvement

Each policy is evaluated by a value function combining a quadratic term with a small set of nonlinear corrections,

$$\widehat{V}_i(\boldsymbol{x}) = \boldsymbol{x}^\top P_i\,\boldsymbol{x} + \boldsymbol{c}_i^\top \boldsymbol{\phi}_{V,\mathrm{nl}}(\boldsymbol{x}), \qquad \widehat{u}_{i+1}(\boldsymbol{x}) = -\tfrac{1}{2}R^{-1}\boldsymbol{g}(\boldsymbol{x})^\top \nabla\widehat{V}_i(\boldsymbol{x}), \tag{17}$$

with $P_i = P_i^\top$. The nonlinear basis $\boldsymbol{\phi}_{V,\mathrm{nl}}$ consists of the monomials $\{x_1^4,\ x_1^3x_2,\ x_1^2x_2^2,\ x_1^2x_3,\ x_1^2x_2x_3,\ x_1^3x_3\}$, all of total degree three or four; they vanish at the origin together with their first derivatives and contain no constant or linear terms, as required. They are chosen to reflect the

process structure: the Duffing potential contributes the quartic x_1^4, and the cross terms capture the coupling of the cubic restoring force into the velocity/acceleration channels. Collecting the six quadratic monomials and the six nonlinear ones into the stacked feature vector $\Phi_V(\boldsymbol{x}) = [x_1^2,\, x_1x_2,\, x_1x_3,\, x_2^2,\, x_2x_3,\, x_3^2,\, \boldsymbol{\phi}_{V,\mathrm{nl}}^\top]^\top$, the value function is linear in its 12 coefficients, $\widehat{V}_i(\boldsymbol{x}) = \boldsymbol{w}_i^\top \Phi_V(\boldsymbol{x})$ with $\boldsymbol{w}_i = [\mathrm{vech}(P_i); \boldsymbol{c}_i]$. Since $\boldsymbol{g} = [0, 0, c]^\top$ is constant, the improved policy is likewise linear in the value weights, $\widehat{u}_{i+1}(\boldsymbol{x}) = -\frac{1}{2R}\,\zeta(\boldsymbol{x})^\top \boldsymbol{w}_i$ with $\zeta(\boldsymbol{x}) = c\,\partial\Phi_V/\partial x_3$.

4.4 Relation to the policy approximation presented in the course

In the course's actor–critic formulation [6] the control policy is approximated by a *separate* network (the actor) with its own weights, tuned alongside the critic. Here, by contrast, the policy is *not* independently parameterized: it is obtained analytically as the gradient of the single value-function approximation, $\widehat{u}_{i+1} = -\frac{1}{2}R^{-1}\boldsymbol{g}^\top \nabla \widehat{V}_i$. Consequently there is one set of unknowns $(\boldsymbol{w}_i)$ instead of two, the improved policy is exactly consistent with $\widehat{V}_i$, and no separate actor-tuning law or its persistence-of-excitation conditions are needed — the excitation requirement falls entirely on the off-policy data.

4.5 Off-policy equations, numerical solution, policy iterations, and validation

The off-policy equation follows in three steps. Along the *measured* (behavior) trajectory, driven by the exciting input u rather than by the evaluated policy u_i, the chain rule gives

$$\frac{d}{dt}\widehat{V}_i(\boldsymbol{x}) = \nabla \widehat{V}_i^\top (\boldsymbol{f} + \boldsymbol{g}u) = \underbrace{\nabla \widehat{V}_i^\top (\boldsymbol{f} + \boldsymbol{g}u_i)}_{\text{policy evaluation}} + \nabla \widehat{V}_i^\top \boldsymbol{g}\,(u - u_i). \tag{18}$$

The first term is replaced by the policy-evaluation (Lyapunov) identity of u_i, $\nabla \widehat{V}_i^\top (\boldsymbol{f}+\boldsymbol{g}u_i) = -(\boldsymbol{x}^\top Q \boldsymbol{x} + R\,u_i^2)$, and in the second term $\nabla \widehat{V}_i^\top \boldsymbol{g} = -2R\,\widehat{u}_{i+1}$ by the policy-improvement definition (17) — so the unknown drift $\boldsymbol{f}$, which appears only inside the evaluated directional derivative, is eliminated. Integrating over a window $[t,\, t{+}T]$ yields the *off-policy Bellman equation*, which contains only measured signals:

$$\widehat{V}_i(\boldsymbol{x}(t{+}T)) - \widehat{V}_i(\boldsymbol{x}(t)) = -\int_t^{t+T} (\boldsymbol{x}^\top Q \boldsymbol{x} + R\,u_i^2) d\tau \;-\; 2\int_t^{t+T} R\,\widehat{u}_{i+1}\,(u - u_i)\,d\tau. \tag{19}$$

Substituting the parameterizations, each window j yields one linear equation in $\boldsymbol{w}_i$,

$$\Big[\Delta\Phi_{V,j} - \int_j \zeta(\boldsymbol{x})\,(u - u_i)\,d\tau\Big]^\top \boldsymbol{w}_i = -\int_j (\boldsymbol{x}^\top Q \boldsymbol{x} + R\,u_i^2) d\tau, \tag{20}$$

with $\Delta\Phi_{V,j} = \Phi_V(\boldsymbol{x}(t_j{+}T)) - \Phi_V(\boldsymbol{x}(t_j))$ and u_i the current policy evaluated on the stored states ($u_0 = 0$); the window integrals are evaluated by the trapezoidal rule on the 1 ms samples. Stacking the 400 windows gives an overdetermined 400×12 system solved by the pseudoinverse (least squares); the regressor has full column rank (12) with condition number $\approx 1.1 \times 10^4$, i.e. the excitation is adequate. The same data build every iteration's regressor. Iteration stops when $\|\boldsymbol{w}_i - \boldsymbol{w}_{i-1}\| < 10^{-4}$.

The stopping criterion is met after 9 iterations (Fig. 10); the eigenvalues of P_i and the value coefficients visibly settle by the fourth iteration, every P_i is positive definite throughout (so the quadratic part of the value function is locally positive definite as required), and the accumulated cost of the *intermediate* policies (Fig. 10, rightmost; evaluated by rollouts from $\boldsymbol{x}_0^{(1)}$ and $\boldsymbol{x}_0^{(4)}$) settles to its final value by the fifth iteration. The transient rise of J_T at the first iteration is the signature of *approximate* policy evaluation on a finite basis — monotone improvement is guaranteed only for exact evaluation — and it disappears as the coefficients converge. As a sanity check, near the origin the nonlinear terms vanish and the identified quadratic part

matches the algebraic-Riccati (LQR) solution of the linearized system to 1.1%. The learned policy is validated against the initial policy $u_0 = 0$ from four initial conditions spanning the operating region,

$$\begin{aligned} \boldsymbol{x}_0^{(1)} &= [0.6,\ 0,\ 0]^\top, & \boldsymbol{x}_0^{(2)} &= [-0.5,\ 1.0,\ 0]^\top, \\ \boldsymbol{x}_0^{(3)} &= [0.4,\ -0.8,\ 2.0]^\top, & \boldsymbol{x}_0^{(4)} &= [0.7,\ 0.5,\ -1.0]^\top \end{aligned} \tag{21}$$

(units m, m/s, m/s^2; $\boldsymbol{x}_0^{(4)}$ is also the Part-4 demonstration case). Figure 11 shows the time histories of *all three states and the control input* for three of them: the learned policy regulates the lightly-damped process to the origin in about 1 s without the sustained oscillations of the uncontrolled response, at a bounded control effort ($|u| \leq 24$ V), and reduces the accumulated cost J_T by 41–56% across all four initial conditions (Fig. 9, right). The identified coefficients $\boldsymbol{w}$ are exported and used, frozen, in the Pico implementation (Section 5).

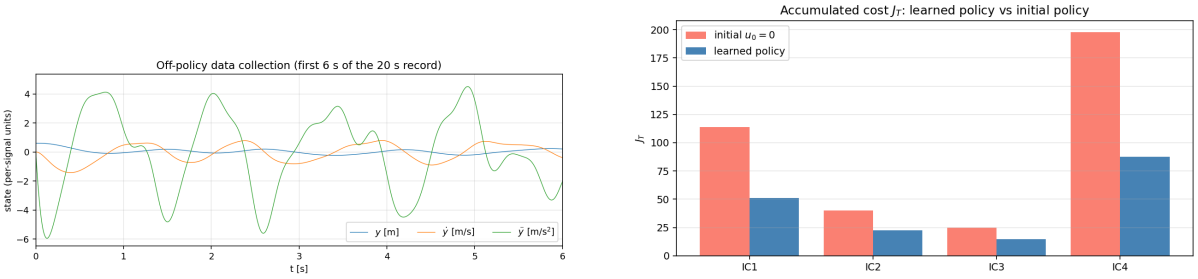

Figure 9: Left: off-policy data collection keeps all three states bounded (first 6 s of the 20 s record; per-signal units m, m/s, m/s^2). Right: accumulated cost J_T of the learned policy vs. the initial policy $u_0 = 0$ over the four initial conditions of (21).

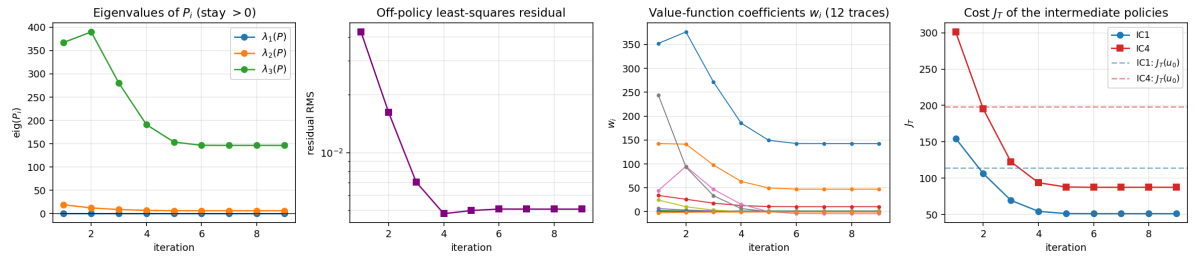

Figure 10: Policy-iteration convergence: the eigenvalues of P_i stay positive, the off-policy least-squares residual settles, all 12 value-function coefficients w_i converge, and the accumulated cost J_T of the intermediate policies (rollouts from $\boldsymbol{x}_0^{(1)}, \boldsymbol{x}_0^{(4)}$; dashed lines: the cost of the initial policy $u_0 = 0$) reaches its final value by the fifth iteration; the stopping criterion is met after 9 iterations.

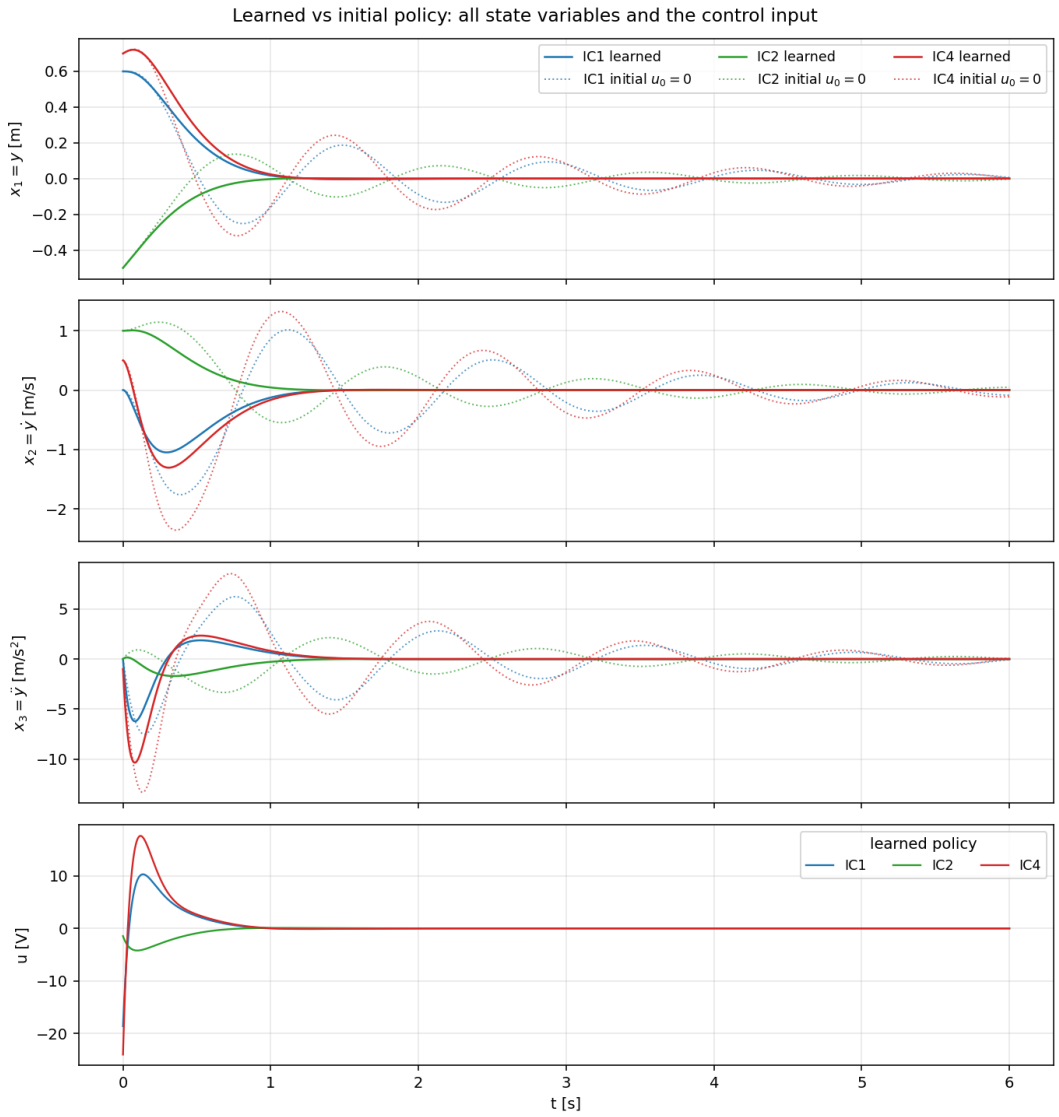

Figure 11: Validation of the learned policy against the initial policy $u_0 = 0$ from the initial conditions $\boldsymbol{x}_0^{(1)}, \boldsymbol{x}_0^{(2)}, \boldsymbol{x}_0^{(4)}$ of (21): time histories of all three state variables (learned: solid; initial: dotted) and the learned-policy control input (bottom).

4.6 Proposed extension to a tracking problem

The method extends conceptually to tracking a reference $\boldsymbol{x}_d(t)$ by regulating the tracking error $\boldsymbol{\eta} = \boldsymbol{x} - \boldsymbol{x}_d$. Augmenting the state with the (exogenous) reference dynamics and defining the cost $J = \int_0^\infty (\boldsymbol{\eta}^\top Q \boldsymbol{\eta} + R\,u^2)\,dt$, the same off-policy machinery applies to the augmented system: the value function and policy become functions of $(\boldsymbol{x}, \boldsymbol{x}_d)$, the value basis is augmented with error monomials, and the off-policy data must additionally excite the reference. Because f is unknown, the exact feedforward input that cancels the drift along $\boldsymbol{x}_d$ is not directly available, so exact asymptotic tracking cannot be guaranteed; instead the tracking error converges to a bounded neighbourhood of zero whose size depends on the value-basis approximation error and the richness of the data. Exact tracking would follow only under the additional assumption that the feedforward term lies in the span of the chosen basis. The required deliverable here is this formulation; it is not implemented.

5 Raspberry Pi Pico Implementation

Both controllers are implemented in MicroPython on the Raspberry Pi Pico (RP2040, dual-core Cortex-M0+). The board runs MicroPython v1.24.0 built with `ulab` 6.12 (single-precision vector arithmetic), with the system clock at the officially supported fast setting of 200 MHz. No external plant is used: the nonlinear process is simulated on the Pico itself, with a fixed allocation of the two cores. The MicroPython modules (`plant_core1.py`, `pico_control.py`, `params.py`) are written once and run both on the board and, through a NumPy fallback, on a desktop, so the on-chip results can be checked against the computer results of Sections 3 and 4. All results in this section are measured **on the physical board** (captured over USB by `pico_experiment.py` and post-processed by `plot_hw_results.py`); the desktop emulation of the same two-core scheme (`desktop_sim.py`) is used as the reference to overlay against.

5.1 Two-core architecture and real-time process simulation

The core allocation is shown in Fig. 12. The **secondary core** simulates the complete nonlinear process: a paced real-time loop performs exactly one RK4 integration step of $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}u$ every $dt = 2$ ms, holding u constant between controller updates (ZOH) and catching up if a step is transiently delayed. The step $dt = 2$ ms is amply small relative to the process dynamics — the fastest closed-loop mode has a time constant above 0.1 s ($|\mathrm{Re}(s)| \leq 8\ \mathrm{s}^{-1}$), so RK4 sees $\gtrsim 60$ steps per time constant. The drift $\boldsymbol{f}$ is evaluated *only* here, so the controller never accesses it. A paced loop is used rather than a `machine.Timer` for a reason discovered during hardware bring-up: on the RP2040 port, MicroPython soft-timer callbacks are always dispatched on core 0 regardless of which core created the timer — this was verified on the board by reading the SIO `CPUID` register from inside the callbacks — whereas code inside a `_thread` task genuinely executes on core 1. The **main core** runs the active controller from a hardware timer with period $T_s = N\,dt = 10$ ms ($N = 5$): each interrupt reads the latest full state $\boldsymbol{x}$ from the secondary core, computes the control input u (and, for Branch A, adapts the RBF weights), and publishes u back. The only variables exchanged are the state $\boldsymbol{x}$ (secondary core $\rightarrow$ main core) and the scalar input u (main core $\rightarrow$ secondary core), both under a lock; the plant holds $\boldsymbol{x}_0$ until the first u is published, so the on-chip run and the computer reference start aligned.

The sampling period $T_s = 10$ ms (100 Hz) was chosen from *on-chip* measurements. The initial design value $T_s = 5$ ms, selected in Section 3.4, assumed ample headroom for the controller computation; the first on-chip measurement (Section 5.2) showed that at 216 centers the RBF update does not fit that budget on the FPU-less M0+. Doubling the period to 100 Hz restores a comfortable margin between the worst-case update time and T_s while still sitting deep inside the stable region of the discrete design — the Section 3.4 sweep is unchanged down to 20 Hz, with degradation only at 15 Hz.

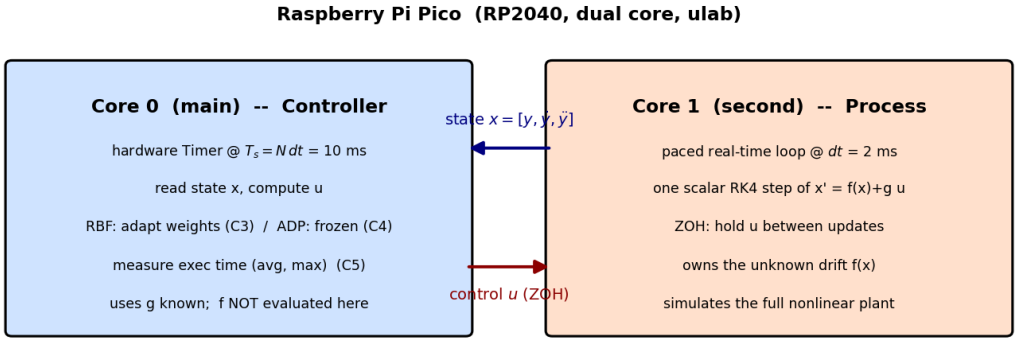

Figure 12: Two-core real-time architecture: the secondary core simulates the process (owning $\boldsymbol{f}$) in a paced loop at dt, the main core runs the controller from a hardware timer at T_s; they exchange only $\boldsymbol{x}$ and u. The labels C3–C5 refer to the assignment's implementation work items (RBF controller, ADP controller, execution-time measurement).

5.2 Adaptive RBF-NN implementation

On the main core the adaptive controller (`RBFController`) evaluates the normalized RBF vector, computes u from (11), updates the 216 weights by one forward-Euler step of (14) with projection, and publishes u to the process core — the same per-sample order as the discrete design of Section 3.3.

Implementing this within the real-time budget required attention the desktop proxy could not provide: the desktop NumPy timing ($\approx 17\,\mu$s per RBF update) suggested a huge margin, but the first on-chip measurement gave 11.9 ms — the RP2040 has no FPU, and at 216 centers both the vectorised exponential and every 216-element `ulab` operation (≈ 0.3 ms each) are expensive. Three optimizations, all preserving the numerical design exactly, bring the update to 2.9 ms:

1. **Separable RBF evaluation.** The centers form a $6 \times 6 \times 6$ Cartesian grid with a common normalized width, so the Gaussian vector factorizes, $e^{-(d_1^2+d_2^2+d_3^2)} = e^{-d_1^2}\, e^{-d_2^2}\, e^{-d_3^2}$: three *six*-element exponentials are combined by two broadcast (outer-product) multiplications into the 216-element vector $\boldsymbol{r}$, replacing the 216-element exponential and most of the long vector operations. The result is identical to machine precision ($\max|\Delta\phi| < 4 \times 10^{-16}$ verified against the direct formula (10)).

2. **Folded normalization and in-place updates.** The normalization $\boldsymbol{\phi} = \boldsymbol{r}/\Sigma_j r_j$ is folded into the two scalar results ($\widehat{W}^\top \boldsymbol{\phi}$ and the weight-update step size, each scaled by $1/\Sigma_j r_j$), and the remaining vector operations are performed in-place, minimizing per-update allocations.

3. **Scalar plant step and deterministic garbage collection.** The length-3 RK4 step is written in scalar arithmetic compiled with the native emitter (0.54 ms instead of 2.9 ms of small-vector `ulab` calls), and the MicroPython garbage collector is invoked explicitly every 10 controller ticks, immediately *after* u is published — collection then happens at a controlled point and never delays a control computation unpredictably.

Fig. 13 shows the 30 s on-chip tracking run. The steady-state RMS tracking error measured on the board is 9.96×10^{-3} m — matching the desktop two-core emulation (9.98×10^{-3} m) and the fine-rate discrete design of Section 3.4 (9.0–9.1×10^{-3} m). The board trajectory deviates from the desktop emulation by at most 3.1×10^{-3} m, most of it during the adaptation transient

(the two runs integrate the weight dynamics in `float32` vs. `float64`); after convergence the curves are indistinguishable (right panel).

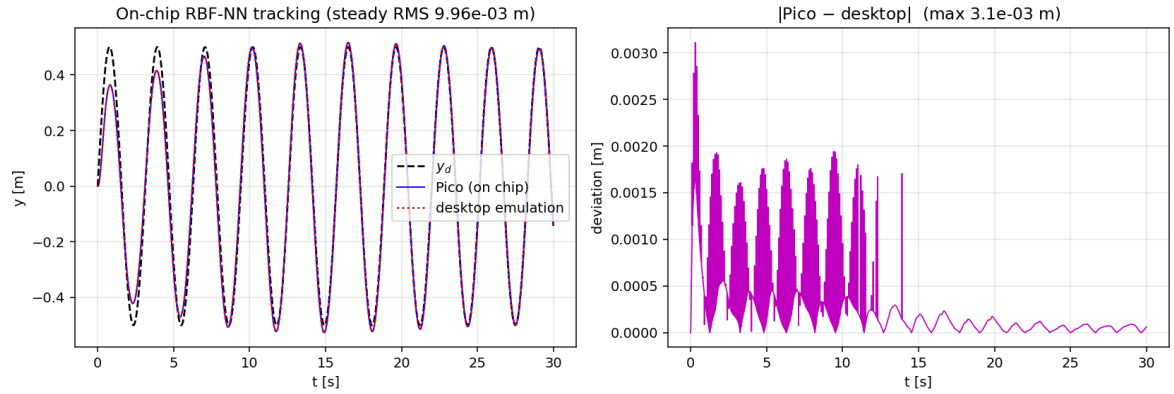

Figure 13: On-chip adaptive RBF-NN tracking (30 s). Left: measured Pico output vs. y_d and the desktop two-core emulation. Right: absolute Pico–desktop deviation.

5.3 ADP final-policy implementation

The optimal-regulation controller (`ADPController`) carries the *frozen* value-function coefficients $\boldsymbol{w}$ identified in Section 4 and, at each update, evaluates the closed form $u = -\frac{1}{2R}\zeta(\boldsymbol{x})^\top \boldsymbol{w}$. Run on the board from $\boldsymbol{x}_0 = [0.7, 0.5, -1]^\top$ (m, m/s, m/s^2) — the fourth validation case of (21) — the measured response coincides with the finely-sampled desktop rollout to $\approx 1.6 \times 10^{-3}$ m at worst (Fig. 14), the small residual being the 10 ms ZOH of the on-chip controller versus the continuous desktop reference.

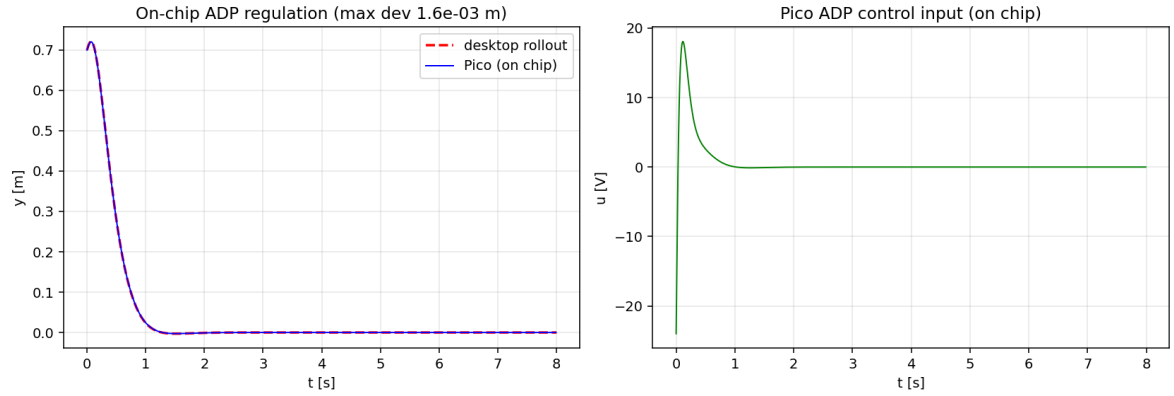

Figure 14: On-chip ADP regulation (8 s). Left: measured Pico output vs. the desktop rollout (near-exact agreement). Right: the on-chip control input.

5.4 Execution-time measurement and comparison with computer results

The main core measures the wall-clock time of every controller update with the microsecond ticks counter. Two intervals are recorded per tick: the *update* interval, from before the state read through the control computation and the weight update, and the *full-tick* interval, which additionally includes the transfer of u back to the process core (a lock-protected scalar store), the logging, and any scheduled garbage collection — so the transfer of the control input is bounded by the measured full tick. The results below cover the 3000 (RBF) and 800 (ADP) updates of the two runs, excluding the cold first tick of each (retained in the submitted captures: 3.2 ms RBF and 0.8 ms ADP — both also below T_s):

Controller update	average	maximum	T_s	margin (T_s/ max)
Adaptive RBF-NN (C3)	2.92 ms	2.99 ms	10 ms	3.3×
ADP final policy (C4)	0.56 ms	0.61 ms	10 ms	16×

Table 3: Measured on-chip controller-update execution times (RP2040 @ 200 MHz, `ulab`).

Both are far below $T_s = 10$ ms (Fig. 15, left), so the real-time constraint $\max(t_{\text{exec}}) < T_s$ is met with a wide margin. The full tick — including the transfer of u, the logging, and the periodic garbage collection — ran ≈ 0.6 ms above the update on ordinary ticks and reached 8.4 ms at its busiest (GC) tick, still under T_s (right panel: the GC ticks are clearly visible every 10 updates). The real-time behaviour is also verified globally: the 30 s (RBF) and 8 s (ADP) experiments completed in 30.05 s and ≈ 8.0 s of wall time with 15 018 and 3 994 plant steps respectively ($dt = 2$ ms), i.e. the controller never missed its period and the plant loop stayed on its long-run schedule; the worst *transient* plant-step delay was 3.5 ms (a GC pause on the main core briefly blocking the plant core's allocation), absorbed by the paced loop's catch-up within two steps. Compared with the desktop proxy ($\approx 17\,\mu$s RBF, $\approx 3\,\mu$s ADP), the RP2040 is roughly two orders of magnitude slower — the reason the on-chip measurement, not the desktop proxy, must set the sampling period, as discussed in Section 5.1.

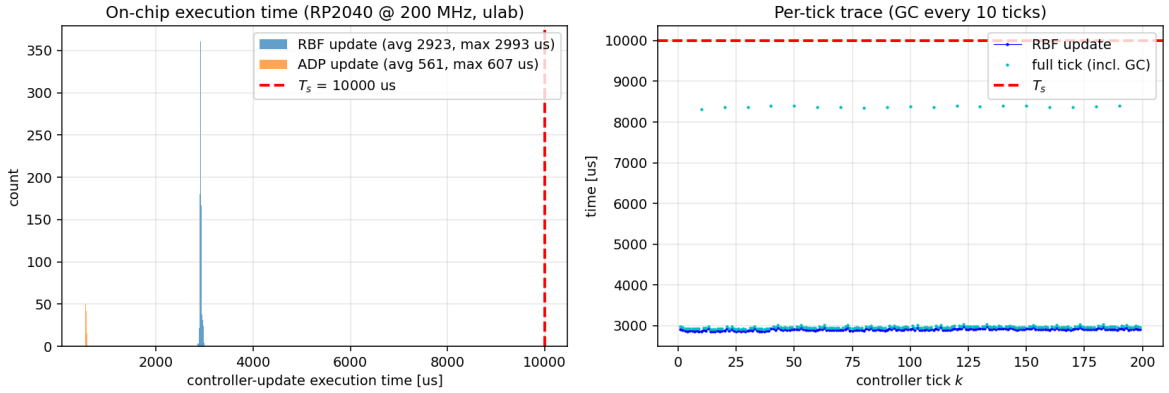

Figure 15: Measured on-chip execution times (RP2040 @ 200 MHz, `ulab`). Left: histogram of the per-update controller execution time for both controllers against T_s. Right: per-tick trace of the RBF run — update time and full tick including the deterministic GC every 10 ticks.

6 Conclusions and Limitations

Two model-free nonlinear controllers were developed for a single Duffing mass-spring-damper process with a first-order actuator and implemented in real time on a Raspberry Pi Pico.

- **Process (Part 1).** The plant was cast in control-affine form with a known input field and an unknown drift, shown to have a unique origin equilibrium that is globally asymptotically stable under zero input by an energy-based Lyapunov argument, and shown to be strongly nonlinear over the operating range (the cubic force reaches the linear force at $y \approx 0.7$ m).
- **Adaptive RBF-NN tracking (Branch A).** A normalized-RBF network learned the unknown drift online; a Lyapunov-based weight law with projection gave a uniformly ultimately bounded tracking error that settled to $\approx 2\%$ of the reference amplitude. The emulation (discrete) realization matched the continuous design down to 20 Hz and degraded at 15 Hz.

- **Off-policy ADP regulation (Branch B).** Policy iteration on a value function $x^\top P x + c^\top \phi_{\mathrm{nl}}$, solved from off-policy data by least squares without the drift, converged in a few iterations with positive-definite P_i; the identified quadratic part matched the LQR/Riccati solution near the origin to 1.1%, and the learned policy cut the accumulated cost by 41–56% over the initial policy.
- **Real-time Pico (Part 4).** The two-core scheme (process on the secondary core at 500 Hz, controller on the main core at 100 Hz, `ulab` at 200 MHz) was run and measured *on the physical board*: the on-chip tracking RMS (9.96×10^{-3} m) matches the computer results, the ADP response coincides with the desktop rollout to $\approx 1.6 \times 10^{-3}$ m, and the measured worst-case update times (2.99 ms RBF, 0.61 ms ADP) satisfy the real-time constraint with a 3.3× margin on the controller update (busiest full tick, including the scheduled garbage collection: 8.4 ms, still below T_s), after separability-based optimization of the RBF evaluation.

The two controllers illustrate complementary model-free paradigms on the *same* process: Branch A learns the drift to *track* a trajectory, while Branch B learns a value function to *regulate* optimally from data.

Limitations. The adaptive tracking result is boundedness, not exact tracking: a residual error persists, set by the RBF approximation error, and the online weights converge only where the trajectory is exciting. The ADP value function is a finite polynomial approximation, so the learned policy is optimal only locally, within the operating region spanned by the data; its benefit over the do-nothing policy shrinks for heavily damped plants. Both designs assume the full state is measured and the input gain is known. Finally, the on-chip experiment still *simulates* the process (on the secondary core); driving a physical plant would add sensing noise, quantization and actuator dynamics that the present setup deliberately excludes.

References

[1] J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*. Englewood Cliffs, NJ: Prentice-Hall, 1991.

[2] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 2002.

[3] R. M. Sanner and J.-J. E. Slotine, "Gaussian networks for direct adaptive control," *IEEE Transactions on Neural Networks*, vol. 3, no. 6, pp. 837–863, 1992.

[4] F. L. Lewis, D. Vrabie, and V. L. Syrmos, *Optimal Control*, 3rd ed. Hoboken, NJ: Wiley, 2012.

[5] Y. Jiang and Z.-P. Jiang, "Computational adaptive optimal control for continuous-time linear systems with completely unknown dynamics," *Automatica*, vol. 48, no. 10, pp. 2699–2704, 2012.

[6] K. G. Vamvoudakis and F. L. Lewis, "Online actor–critic algorithm to solve the continuous-time infinite horizon optimal control problem," *Automatica*, vol. 46, no. 5, pp. 878–888, 2010.

[7] G. F. Franklin, J. D. Powell, and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, 3rd ed. Reading, MA: Addison-Wesley, 1998.

A Code Listings

The key algorithmic sources are listed below. The complete, reproducible archive (all Python and MicroPython files, parameters, and generated data) accompanies the report; see `README.md` for how to regenerate every figure.

A.1 Shared process model `plant.py`

```
"""
Shared nonlinear process for the Mechatronics Systems final project.

Physical process (reused from HW1, per the project spec's permission to continue
with the homework process).  A mass on a Duffing (cubic) spring with linear
viscous damping, driven by a first-order force actuator:

    m y'' + c1 y' + k1 y + k3 y^3 = F          (Duffing mass-spring-damper)
    tau F' + F = ka u                          (first-order force actuator)

State (companion form used by BOTH controllers), x = [y, y', y''] = [x1, x2, x3]:

    x1' = x2
    x2' = x3
    x3' = -a1 x3 - a2 x2 - a3 x1 - b1 x1^3 - b2 x1^2 x2 + c u

i.e. the control-affine form required by the project (Eq. 1):

    x' = f(x) + g(x) u ,     g(x) = [0, 0, c]^T  (CONSTANT -> known to controller)

with f(x) the (smooth, nonlinear) drift.  The scalar drift in the input channel

    D(x) = a1 x3 + a2 x2 + a3 x1 + b1 x1^3 + b2 x1^2 x2        (so f3 = -D(x))

is the "unknown nonlinear function" that Branch A approximates by an RBF network.

f(0) = 0 (spec 1.5 satisfied: the regulation equilibrium is already the origin,
because a3 = k1/(tau m) != 0 ties the mass to ground -> unique equilibrium).

Lyapunov analysis of the zero-input system x' = f(x) is cleanest in the physically
equivalent coordinates z = [y, y', F], related to x by the GLOBAL diffeomorphism

    F = m x3 + c1 x2 + k1 x1 + k3 x1^3   (<->   x3 = (F - c1 x2 - k1 x1 - k3 x1^3)/m)

in which the zero-input dynamics are  z1'=z2, z2'=(z3 - c1 z2 - k1 z1 - k3 z1^3)/m,
z3'=-z3/tau, and the mechanical energy + actuator term is a Lyapunov function
(see part1_analysis.py).

Coefficients (derived from the physical parameters, identical to HW1):
    a1 = c1/m + 1/tau ,  a2 = k1/m + c1/(tau m) ,  a3 = k1/(tau m) ,  c = ka/(tau m)
    b1 = k3/(tau m)   (regressor y^3) ,   b2 = 3 k3/m   (regressor y^2 y')
"""

import numpy as np

# ----- physical parameters -------------------------------------------------
# Structure (Duffing spring + first-order actuator) and the mass/stiffness/actuator
# gain (m, k1, ka) are those of HW1.  Three parameters are re-tuned for the project,
# as the spec permits ("the model may be reformulated or adapted"): a slower, more
# realistic actuator (tau: 0.05 -> 0.2 s), a stronger cubic stiffness (k3: 5 -> 40),
# and lighter damping (c1: 4 -> 1).  The result is a process that is (i) STRONGLY
# nonlinear at a moderate amplitude -- the cubic force k3 y^2/k1 is 50% of the linear
# force already at y = 0.5 m; (ii) lightly damped, so the zero-input response is
# oscillatory and the optimal regulator (Branch B) has a clear benefit over doing
# nothing; while (iii) keeping the unknown drift of moderate magnitude, which makes
# the RBF and ADP designs well conditioned and portable to the Pico numeric range.
m, c1, k1 = 1.0, 1.0, 20.0      # mass [kg], linear damping [N s/m], linear stiffness [N/
    m]
tau, ka = 0.2, 1.0              # actuator time constant [s], actuator gain [N/V]
k3 = 40.0                       # cubic (Duffing) stiffness [N/m^3]

# ----- derived linear jerk coefficients ------------------------------------
```

```
a1 = c1 / m + 1.0 / tau                  # 6
a2 = k1 / m + c1 / (tau * m)             # 25
a3 = k1 / (tau * m)                      # 100   (!= 0  -> unique equilibrium at origin)
c = ka / (tau * m)                       # 5     (input gain: g = [0,0,c])

# ----- derived nonlinear jerk coefficients ----------------------------------
b1 = k3 / (tau * m)             # 200   coefficient of y^3
b2 = 3.0 * k3 / m               # 120   coefficient of y^2 y'  (cross term)

# constant input vector g(x) = g  (control-affine, g known to the controller)
G = np.array([0.0, 0.0, c])


def drift_D(x):
    """Scalar nonlinear drift in the input channel: x3' = c u - D(x).
    This is the 'unknown' function the RBF network (Branch A) approximates."""
    x1, x2, x3 = x[0], x[1], x[2]
    return a1 * x3 + a2 * x2 + a3 * x1 + b1 * x1 ** 3 + b2 * x1 ** 2 * x2


def f(x):
    """Drift field f(x) of the control-affine model x' = f(x) + g u."""
    x1, x2, x3 = x[0], x[1], x[2]
    return np.array([x2, x3, -drift_D(x)])


def g(x=None):
    """Known input field g(x) = [0,0,c] (constant)."""
    return G


def dynamics(x, u):
    """Full control-affine vector field x' = f(x) + g u."""
    return f(x) + G * u


def rk4_step(x, u, dt):
    """One classical RK4 integration step with ZOH input u (held over dt).
    This is the exact integrator ported to the Pico secondary core (Part 4)."""
    k1_ = dynamics(x, u)
    k2_ = dynamics(x + 0.5 * dt * k1_, u)
    k3_ = dynamics(x + 0.5 * dt * k2_, u)
    k4_ = dynamics(x + dt * k3_, u)
    return x + (dt / 6.0) * (k1_ + 2 * k2_ + 2 * k3_ + k4_)


# ----- linearization about the origin (for the M4 comparison) ---------------
# Dropping the nonlinear terms b1 x1^3 and b2 x1^2 x2 gives the Jacobian A:
A_lin = np.array([[0.0, 1.0, 0.0],
                  [0.0, 0.0, 1.0],
                  [-a3, -a2, -a1]])
B_lin = np.array([0.0, 0.0, c])


def f_linear(x):
    """Linearized drift A_lin x (zero-input) for the nonlinear-vs-linear study."""
    return A_lin @ np.asarray(x, dtype=float)


def dynamics_linear(x, u):
    return A_lin @ np.asarray(x, dtype=float) + B_lin * u


# ----- physical actuator force + energy-based Lyapunov function (Part 1) -----
def force_F(x):
    """Physical actuator force F as a function of the companion state x.
    z = [y, y', F] is a global diffeomorphism of x = [y, y', y'']."""
    x1, x2, x3 = x[0], x[1], x[2]
    return m * x3 + c1 * x2 + k1 * x1 + k3 * x1 ** 3


def lyapunov_V(x, gamma):
    """Lyapunov candidate for the zero-input system, in physical coordinates:
```

```
        V = 1/2 m y'^2 + 1/2 k1 y^2 + 1/4 k3 y^4 + 1/2 gamma F^2 .
    Positive definite and radially unbounded (k3>0)."""
    x1, x2 = x[0], x[1]
    F = force_F(x)
    return 0.5 * m * x2 ** 2 + 0.5 * k1 * x1 ** 2 + 0.25 * k3 * x1 ** 4 + 0.5 * gamma * F
     ** 2


def lyapunov_Vdot(x, gamma):
    """Time derivative of V along the ZERO-INPUT flow x' = f(x).
    In physical coordinates this collapses to the quadratic form
        Vdot = -c1 y'^2 - (gamma/tau) F^2 + y' F ,
    negative definite in (y', F) whenever gamma > tau/(4 c1)."""
    x2 = x[1]
    F = force_F(x)
    return -c1 * x2 ** 2 - (gamma / tau) * F ** 2 + x2 * F


# gamma making Vdot negative (semi)definite: need c1*(gamma/tau) - 1/4 > 0
GAMMA_LYAP = tau / (4.0 * c1) * 4.0     # = tau/c1  (comfortably > tau/(4 c1))


if __name__ == "__main__":
    print("Control-affine Duffing jerk plant  x' = f(x) + g u ,  g = [0,0,c]")
    print(f"  a1={a1:.1f} a2={a2:.1f} a3={a3:.1f} c={c:.1f} | b1={b1:.1f}(y^3) b2={b2:.1f
    }(y^2 y')")
    print(f"  f(0) = {f(np.zeros(3))}  (origin is an equilibrium)")
    # unique equilibrium check: -a3 x1 - b1 x1^3 = 0 -> x1(a3+b1 x1^2)=0 -> x1=0
    print(f"  a3,b1 > 0  -> a3 + b1 x1^2 > 0  -> origin is the UNIQUE zero-input
    equilibrium")
    # open-loop linear stability (Routh on s^3+a1 s^2+a2 s+a3)
    print(f"  linear open-loop: a1*a2={a1*a2:.0f} > a3={a3:.0f} -> {a1*a2 > a3} (Hurwitz)
    ")
    print(f"  linear eigenvalues: {np.round(np.linalg.eigvals(A_lin), 4)}")
    # Lyapunov spot check on a grid: Vdot <= 0 everywhere, =0 only at y'=F=0
    gamma = GAMMA_LYAP
    print(f"  gamma_lyap = {gamma:.4f}  (need > tau/(4 c1) = {tau/(4*c1):.4f})")
    rng = np.random.default_rng(0)
    Xs = rng.uniform(-3, 3, size=(20000, 3))
    vdots = np.array([lyapunov_Vdot(x, gamma) for x in Xs])
    print(f"  Vdot over 20000 random states in [-3,3]^3: max = {vdots.max():.3e} (<=0
    expected)")
```

A.2 Branch A – adaptive RBF-NN controller `rbf_adaptive.py`

```
"""
Branch A -- adaptive RBF neural-network TRACKING controller (report Section 3,
grading A1-A3, A5-continuous).

Plant (control-affine, relative degree 3):   x' = f(x) + g u ,  y = x1 .
Unknown drift D(x) (Eq. drift in plant.py) approximated by an RBF network
    Dhat(x) = What^T phi(x) ,   phi = Gaussian RBFs on a grid (config.rbf_*).

Tracking errors  xi = [e, e', e''] ,  e = y - y_d .  With the feedback-linearizing
control (c and phi known; What adapted online)
    u = (1/c) [ What^T phi(x) + y_d''' - l2 e'' - l1 e' - l0 e ] ,
the error dynamics are
    xi' = Lambda xi + b ( What^T phi - D ) ,   b = [0,0,1]^T ,
with Lambda the Hurwitz companion matrix of the desired error poles (config.ERR_POLES).

Lyapunov design.  With P = P^T > 0 solving  Lambda^T P + P Lambda = -Q  and
    V = xi^T P xi + (1/gamma) Wtil^T Wtil ,   Wtil = What - W* ,
the weight-adaptation law is the Lyapunov gradient law with PROJECTION,
    What' = Proj( -gamma ( b^T P xi ) phi(x) ),   |w_i| <= W_MAX,
which gives  V' <= -xi^T Q xi - 2 (b^T P xi) eps,  i.e. the tracking error and the
weights are uniformly ultimately bounded, with the residual set fixed by the RBF
approximation error eps.  (weight_dot also supports sigma-modification as an
alternative robustness term; the design in the report uses projection only,
config.SIGMA_MOD = 0.)  The nominal controller (A1) replaces What^T phi by the true
D(x) and yields the exact linear error dynamics xi' = Lambda xi.
"""
```

```
27
28  import os
29  import numpy as np
30  from scipy.integrate import solve_ivp
31  from scipy.linalg import solve_continuous_lyapunov
32  import matplotlib
33  matplotlib.use("Agg")
34  import matplotlib.pyplot as plt
35
36  import plant as P
37  import config as C
38
39  # ----- error-dynamics companion matrix, Lyapunov P, RBF centers -------------
40  L2, L1, L0 = C.LAM                          # gains (from poles)
41  LAMBDA = np.array([[0.0, 1.0, 0.0],
42                     [0.0, 0.0, 1.0],
43                     [-L0, -L1, -L2]])
44  P_LYAP = solve_continuous_lyapunov(LAMBDA.T, -C.LYAP_Q)   # Lambda^T P + P Lambda = -Q
45  b_vec = np.array([0.0, 0.0, 1.0])
46  bP = b_vec @ P_LYAP                          # row vector b^T P  (= 3rd row of P)
47  CENTERS = C.rbf_centers()
48
49
50  def _sat(u):
51      return float(np.clip(u, -C.U_MAX, C.U_MAX))
52
53
54  def weight_dot(W, bPxi, phi):
55      """Lyapunov gradient weight-adaptation law with projection to |w_i| <= W_MAX
56      (and optional sigma-modification).  The projection cancels any drive that would
57      push a saturated weight further out, which keeps ||W|| bounded while preserving
58      the Lyapunov inequality.  Reused by the discrete controller and the Pico port."""
59      tau = -C.GAMMA_ADAPT * bPxi * phi - C.GAMMA_ADAPT * C.SIGMA_MOD * W
60      out = tau.copy()
61      out[(W >= C.W_MAX) & (tau > 0)] = 0.0
62      out[(W <= -C.W_MAX) & (tau < 0)] = 0.0
63      return out
64
65
66  def controller(x, W, t, adaptive):
67      """Return (u, xi, phi, Dhat) for the current state/weights."""
68      yd, yd1, yd2, yd3 = C.y_ref(t)
69      e = x[0] - yd
70      ed = x[1] - yd1
71      edd = x[2] - yd2
72      xi = np.array([e, ed, edd])
73      if adaptive:
74          phi = C.rbf_phi(x, CENTERS)
75          Dhat = W @ phi
76      else:
77          phi = None
78          Dhat = P.drift_D(x)                 # nominal: true drift (f known)
79      u = _sat((Dhat + yd3 - L2 * edd - L1 * ed - L0 * e) / P.c)
80      return u, xi, phi, Dhat
81
82
83  def simulate(adaptive, T=20.0, x0=(0.0, 0.0, 0.0), dt_out=2e-3):
84      """Closed-loop simulation. State = [x(3), What(N_RBF)] (weights only if adaptive)."""
85      n = C.N_RBF
86
87      def ode(t, S):
88          x = S[:3]
89          W = S[3:] if adaptive else None
90          u, xi, phi, _ = controller(x, W, t, adaptive)
91          xdot = P.dynamics(x, u)
92          if not adaptive:
93              return list(xdot)
94          Wdot = weight_dot(W, bP @ xi, phi)
95          return list(xdot) + list(Wdot)
96
97      S0 = list(x0) + ([0.0] * n if adaptive else [])
98      te = np.arange(0, T + dt_out / 2, dt_out)
99      sol = solve_ivp(ode, [0, T], S0, t_eval=te, method="RK45", rtol=1e-7, atol=1e-9)
```

```
    x = sol.y[:3]
    t = sol.t
    yd = np.array([C.y_ref(tt)[0] for tt in t])
    # reconstruct control, RBF output and true drift along the trajectory
    u = np.empty(len(t)); Dhat = np.empty(len(t)); Dtrue = np.empty(len(t))
    Wn = np.zeros(len(t))
    for i, tt in enumerate(t):
        W = sol.y[3:, i] if adaptive else None
        ui, _, _, Dh = controller(x[:, i], W, tt, adaptive)
        u[i] = ui; Dhat[i] = Dh; Dtrue[i] = P.drift_D(x[:, i])
        if adaptive:
            Wn[i] = np.linalg.norm(W)
    return dict(t=t, x=x, y=x[0], yd=yd, e=x[0] - yd, u=u,
                Dhat=Dhat, Dtrue=Dtrue, Wnorm=Wn,
                W=(sol.y[3:] if adaptive else None))


# ============================================================================
# Figures (A5, continuous part)
# ============================================================================
def make_figures(da, dn, outdir="images/part2"):
    os.makedirs(outdir, exist_ok=True)

    # tracking: adaptive vs nominal vs reference
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(dn["t"], dn["yd"], "k--", lw=1.6, label=r"$y_d$ (reference)")
    ax.plot(dn["t"], dn["y"], "r:", lw=1.5, label=r"$y$ nominal ($f$ known)")
    ax.plot(da["t"], da["y"], "b-", lw=1.3, label=r"$y$ adaptive (RBF, cold start)")
    ax.set_title("Trajectory tracking: nominal vs adaptive RBF-NN")
    ax.set_xlabel("t [s]"); ax.set_ylabel("y [m]"); ax.legend(loc="upper right"); ax.grid
    (alpha=.3)
    fig.tight_layout(); fig.savefig(f"{outdir}/tracking.png", dpi=140); plt.close()

    # tracking error
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.semilogy(dn["t"], np.abs(dn["e"]) + 1e-18, "r:", lw=1.4, label="nominal")
    ax.semilogy(da["t"], np.abs(da["e"]) + 1e-18, "b-", lw=1.3, label="adaptive")
    ax.set_title(r"Tracking error $|e(t)|$")
    ax.set_xlabel("t [s]"); ax.set_ylabel("|e| [m]"); ax.legend(); ax.grid(alpha=.3,
    which="both")
    fig.tight_layout(); fig.savefig(f"{outdir}/tracking_error.png", dpi=140); plt.close()

    # control input
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(da["t"], da["u"], "b-", lw=1.1, label="adaptive")
    ax.plot(dn["t"], dn["u"], "r:", lw=1.1, label="nominal")
    ax.axhline(C.U_MAX, color="gray", ls=":", lw=.7); ax.axhline(-C.U_MAX, color="gray",
    ls=":", lw=.7)
    ax.set_title("Control input $u(t)$"); ax.set_xlabel("t [s]"); ax.set_ylabel("u [V]")
    ax.legend(); ax.grid(alpha=.3)
    fig.tight_layout(); fig.savefig(f"{outdir}/control.png", dpi=140); plt.close()

    # RBF approximation of the drift
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(da["t"], da["Dtrue"], "k-", lw=1.6, label=r"true drift $D(x)$")
    ax.plot(da["t"], da["Dhat"], "b--", lw=1.3, label=r"RBF estimate $\widehat{D}(x)=\
    widehat{W}^\top\phi$")
    ax.set_title("Online RBF approximation of the unknown drift")
    ax.set_xlabel("t [s]"); ax.set_ylabel(r"drift $D$ [m/s$^3$]"); ax.legend(); ax.grid(
    alpha=.3)
    fig.tight_layout(); fig.savefig(f"{outdir}/rbf_drift.png", dpi=140); plt.close()

    # weight norm
    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(da["t"], da["Wnorm"], "purple", lw=1.4)
    ax.set_title(r"RBF weight norm $\|\widehat{W}(t)\|$ (bounded)")
    ax.set_xlabel("t [s]"); ax.set_ylabel(r"$\|\widehat{W}\|$"); ax.grid(alpha=.3)
    fig.tight_layout(); fig.savefig(f"{outdir}/weights.png", dpi=140); plt.close()

    # full state histories x2, x3 (x1 = y is the tracking figure) -- A5
    fig, ax = plt.subplots(2, 1, figsize=(9, 5.6), sharex=True)
    ydd = np.gradient(da["yd"], da["t"])           # y_d' for visual reference
    ax[0].plot(da["t"], da["x"][1], "b-", lw=1.1, label="adaptive")
```

```
    ax[0].plot(dn["t"], dn["x"][1], "r:", lw=1.1, label="nominal")
    ax[0].plot(da["t"], ydd, "k--", lw=1.0, label=r"$\dot y_d$")
    ax[0].set_ylim(-1.12, 1.5)                          # headroom so the legend clears the
    data
    ax[0].set_ylabel(r"$x_2=\dot y$ [m/s]"); ax[0].legend(loc="upper right", ncol=3); ax
    [0].grid(alpha=.3)
    ax[1].plot(da["t"], da["x"][2], "b-", lw=1.1, label="adaptive")
    ax[1].plot(dn["t"], dn["x"][2], "r:", lw=1.1, label="nominal")
    ax[1].set_ylabel(r"$x_3=\ddot y$ [m/s$^2$]"); ax[1].set_xlabel("t [s]")
    ax[1].legend(loc="upper right"); ax[1].grid(alpha=.3)
    fig.suptitle("State histories under the nominal and adaptive controllers")
    fig.tight_layout(); fig.savefig(f"{outdir}/states.png", dpi=140); plt.close()

    # representative individual weight estimates -- A5
    fig, ax = plt.subplots(figsize=(9, 4))
    Wh = da["W"]
    idx = np.argsort(np.abs(Wh[:, -1]))[-6:][::-1]      # 6 largest final |w_i|
    for i in idx:
        ax.plot(da["t"], Wh[i], lw=1.1, label=rf"$\widehat w_{{{i+1}}}$")
    ax.set_title("Representative RBF weight estimates (6 largest at $t=T$)")
    ax.set_xlabel("t [s]"); ax.set_ylabel(r"$\widehat w_i$"); ax.legend(ncol=3, fontsize
    =9)
    ax.grid(alpha=.3)
    fig.tight_layout(); fig.savefig(f"{outdir}/weights_traces.png", dpi=140); plt.close()


if __name__ == "__main__":
    print("== Branch A: adaptive RBF-NN tracking (continuous) ==")
    print(f"error poles {C.ERR_POLES}, gains (l2,l1,l0)=({L2},{L1},{L0})")
    print(f"P eig = {np.round(np.linalg.eigvalsh(P_LYAP),4)} (>0), RBF centers = {C.N_RBF
    }")

    dn = simulate(adaptive=False, T=30.0)
    da = simulate(adaptive=True, T=30.0)

    def ss_rms(d, frac=0.5):
        m = int((1 - frac) * len(d["t"]))
        return np.sqrt(np.mean(d["e"][m:] ** 2))

    print(f"[nominal ] max|e|={np.max(np.abs(dn['e'])):.3e}  ss-RMS|e|={ss_rms(dn):.3e}")
    print(f"[adaptive] max|e|={np.max(np.abs(da['e'])):.3e}  ss-RMS|e|={ss_rms(da):.3e}")
    m = int(0.5 * len(da["t"]))
    drift_rms = np.sqrt(np.mean((da["Dhat"][m:] - da["Dtrue"][m:]) ** 2))
    print(f"[adaptive] steady RBF drift error RMS = {drift_rms:.3e}, "
          f"final ||W|| = {da['Wnorm'][-1]:.2f}, max|u| = {np.max(np.abs(da['u'])):.1f}")
    make_figures(da, dn)
    print("saved figures -> images/part2/")
```

A.3 Branch B – off-policy ADP `adp_offpolicy.py`

```
r"""
Branch B -- nonlinear OFF-POLICY adaptive dynamic programming (report Section 4,
grading B1-B8).

Optimal regulation of x' = f(x) + g u (f unknown, g known) for the cost
    J = \int_0^inf (x^T Q x + R u^2) dt ,   Q>0, R>0.

Policy iteration (Lewis/Vamvoudakis; Jiang & Jiang off-policy IRL):
  * value  V_i(x)   = x^T P_i x + c_i^T phi_nl(x)                    (Eq. B4-6)
  * policy u_{i+1}  = -(1/2) R^{-1} g(x)^T grad V_i(x)               (Eq. B4-7)

Off-policy Bellman equation along the REAL data (behaviour input u, real trajectory),
which never uses f:
  V_i(x(t+T)) - V_i(x(t)) = -\int_t^{t+T}(x^T Q x + R u_i^2) dtau
                            - 2\int_t^{t+T} R u_{i+1} (u - u_i) dtau .

Writing V_i = w_i . Phi_V(x) with Phi_V = [quadratic monomials ; phi_nl], and using
u_{i+1} = -(1/(2R)) zeta(x)^T w_i with zeta(x) = c * dPhi_V/dx3 (g=[0,0,c]), each data
window j gives ONE linear equation in the single unknown vector w_i:
    [ dPhi_V_j - \int_j zeta (u - u_i) dtau ] . w_i = -\int_j (x^T Q x + R u_i^2) dtau .
With many more windows than unknowns, w_i is found by least squares (pseudoinverse);
```

```
the same data set is reused for every iteration.  u_i is the current policy evaluated
on the stored states (u_0 = 0).
"""

import os
import numpy as np
from scipy.linalg import solve_continuous_are
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

import plant as P
import config as C

R = C.ADP_R
Q = C.ADP_Q
MON = C.ADP_MONOMIALS                       # nonlinear monomial exponents (m,3)
N_NL = MON.shape[0]
N_W = 6 + N_NL                              # 6 quadratic + nonlinear unknowns


# ----- value-function basis Phi_V(x) and its x3-derivative ------------------
def phi_V(x):
    """Full value-function basis: 6 quadratic monomials + nonlinear monomials."""
    x1, x2, x3 = x
    quad = [x1 * x1, x1 * x2, x1 * x3, x2 * x2, x2 * x3, x3 * x3]
    nl = [x1 ** e[0] * x2 ** e[1] * x3 ** e[2] for e in MON]
    return np.array(quad + nl)


def dphi_V_dx3(x):
    """d Phi_V / d x3  (only the x3-dependent terms survive)."""
    x1, x2, x3 = x
    dquad = [0.0, 0.0, x1, 0.0, x2, 2.0 * x3]
    dnl = []
    for e in MON:
        if e[2] == 0:
            dnl.append(0.0)
        else:
            dnl.append(e[2] * x1 ** e[0] * x2 ** e[1] * x3 ** (e[2] - 1))
    return np.array(dquad + dnl)


def zeta(x):
    """zeta(x) = c * dPhi_V/dx3 ; policy u = -(1/(2R)) zeta(x)^T w."""
    return P.c * dphi_V_dx3(x)


def policy(x, w):
    return -(1.0 / (2.0 * R)) * (zeta(x) @ w)


def P_matrix(w):
    """Reconstruct the symmetric quadratic matrix P_i from the first 6 weights."""
    p = w[:6]
    return np.array([[p[0], 0.5 * p[1], 0.5 * p[2]],
                     [0.5 * p[1], p[3], 0.5 * p[4]],
                     [0.5 * p[2], 0.5 * p[4], p[5]]])


# ----- B3: off-policy data generation ---------------------------------------
def generate_data(T=None, dt=None):
    """Apply a bounded, sufficiently exciting behaviour input (u_0=0 + exploration)
    to the TRUE plant and record (t, X, U).  f is used only by the simulator."""
    dt = C.ADP_DT if dt is None else dt
    T = C.ADP_N_WINDOWS * C.ADP_WINDOW if T is None else T
    n = int(round(T / dt))
    X = np.empty((n + 1, 3)); U = np.empty(n + 1)
    x = C.ADP_X0.copy()
    for k in range(n + 1):
        t = k * dt
        u = C.ADP_PROBE_AMP * np.sum(np.sin(C.ADP_PROBE_FREQS * t)) / len(C.
    ADP_PROBE_FREQS)
```

```
        u = float(np.clip(u, -C.U_MAX, C.U_MAX))
        X[k] = x; U[k] = u
        if k < n:
            x = P.rk4_step(x, u, dt)
    return np.arange(n + 1) * dt, X, U


# ----- B6: off-policy policy iteration ---------------------------------------
def policy_iteration(X, U, dt, verbose=True):
    """Off-policy IRL policy iteration reusing the same dataset each step."""
    Wlen = int(round(C.ADP_WINDOW / dt))          # samples per window
    n_win = (len(X) - 1) // Wlen
    # precompute per-sample basis, zeta, stage-cost-without-control
    Phi = np.array([phi_V(x) for x in X])          # (N, N_W)
    Z = np.array([zeta(x) for x in X])             # (N, N_W)
    xQx = np.einsum("ni,ij,nj->n", X, Q, X)        # x^T Q x per sample

    dPhi = np.empty((n_win, N_W))
    for j in range(n_win):
        dPhi[j] = Phi[(j + 1) * Wlen] - Phi[j * Wlen]

    def win_trapz(integrand):
        """Trapezoidal integral of a per-sample (N,...) array over each window."""
        out = []
        for j in range(n_win):
            s = slice(j * Wlen, (j + 1) * Wlen + 1)
            out.append(np.trapz(integrand[s], dx=dt, axis=0))
        return np.array(out)

    w = np.zeros(N_W)                              # value weights (define policy u_1
    after solve)
    history = {"w": [], "P_eig": [], "res": []}
    for k in range(C.ADP_MAX_ITERS):
        # current policy u_k on the stored states (u_0 = 0)
        u_cur = np.zeros(len(X)) if k == 0 else -(1.0 / (2.0 * R)) * (Z @ w)
        # regressor A_j and target b_j for this iteration (reuse same data)
        A = dPhi - win_trapz(Z * (U - u_cur)[:, None])          # (n_win, N_W)
        b = -win_trapz(xQx + R * u_cur ** 2)                    # (n_win,)
        if k == 0 and verbose:
            print(f"  LS regressor: {A.shape[0]}x{A.shape[1]}, rank {np.linalg.
    matrix_rank(A)}, "
                  f"cond {np.linalg.cond(A):.2e}")
        w_new, *_ = np.linalg.lstsq(A, b, rcond=None)
        res = np.linalg.norm(A @ w_new - b) / np.sqrt(n_win)
        delta = np.linalg.norm(w_new - w)
        w = w_new
        Pe = np.linalg.eigvalsh(P_matrix(w))
        history["w"].append(w.copy()); history["P_eig"].append(Pe); history["res"].append
    (res)
        if verbose:
            print(f"  iter {k:2d}: ||dw||={delta:.3e}  residual={res:.3e}  "
                  f"eig(P)={np.round(Pe,3)}  {'PD' if np.all(Pe>0) else 'NOT PD'}")
        if delta < C.ADP_TOL and k > 0:
            break
    return w, history


# ----- B7: validation -- learned policy vs initial policy u0=0 --------------
def rollout(x0, w, T=6.0, dt=1e-3, use_policy=True):
    """Closed-loop rollout; returns (t, X, U, J_T) with J_T the accumulated cost."""
    n = int(round(T / dt))
    X = np.empty((n + 1, 3)); U = np.empty(n + 1)
    x = np.array(x0, float); J = 0.0
    for k in range(n + 1):
        u = float(np.clip(policy(x, w), -C.U_MAX, C.U_MAX)) if use_policy else 0.0
        X[k] = x; U[k] = u
        stage = x @ Q @ x + R * u * u
        J += stage * dt * (0.5 if k in (0, n) else 1.0)
        if k < n:
            x = P.rk4_step(x, u, dt)
    return np.arange(n + 1) * dt, X, U, J


```

```python
if __name__ == "__main__":
    print("== Branch B: off-policy ADP ==")
    print(f"Q=diag{np.diag(Q).tolist()} R={R}; unknowns={N_W} (6 quad + {N_NL} nonlinear)")
    dt = C.ADP_DT
    t, X, U = generate_data()
    print(f"data: T={t[-1]:.1f}s, {len(X)} samples, |x|max per axis={np.round(np.max(np.abs(X),0),3)} "
          f"(box {C.X_BOX[:,1]}), |u|max={np.max(np.abs(U)):.1f}")

    w, hist = policy_iteration(X, U, dt)
    print(f"stopping criterion ||w_i+1 - w_i|| < {C.ADP_TOL} met after {len(hist['w'])} iterations")
    Pfin = P_matrix(w)
    print(f"learned quadratic P=\n{np.round(Pfin,3)}")
    print(f"eig(P) = {np.round(np.linalg.eigvalsh(Pfin),4)}  (all>0 => locally PD)")

    # sanity: compare quadratic part with the LQR/ARE solution of the linearization
    Pare = solve_continuous_are(P.A_lin, P.B_lin.reshape(3, 1), Q, np.array([[R]]))
    print(f"ARE (linearization) P=\n{np.round(Pare,3)}")
    print(f"||P_adp - P_are||/||P_are|| = {np.linalg.norm(Pfin-Pare)/np.linalg.norm(Pare):.3%}")

    # nonlinear value-function coefficients
    print(f"nonlinear coeffs c = {np.round(w[6:],4)}")

    # B7 validation over several initial conditions
    ics = [[0.6, 0, 0], [-0.5, 1.0, 0], [0.4, -0.8, 2.0], [0.7, 0.5, -1.0]]
    print("\n[B7] accumulated cost J_T (learned vs initial u0=0):")
    Jl, J0 = [], []
    for x0 in ics:
        _, _, _, jl = rollout(x0, w, use_policy=True)
        _, _, _, j0 = rollout(x0, w, use_policy=False)
        Jl.append(jl); J0.append(j0)
        print(f"  x0={x0}:  J_learned={jl:8.3f}   J_(u0=0)={j0:8.3f}   improvement={100*(j0-jl)/j0:5.1f}%")

    # ---------------- figures ----------------
    OUT = "images/part3"; os.makedirs(OUT, exist_ok=True)

    # data-collection state stays in the operating box
    fig, ax = plt.subplots(figsize=(9, 3.6))
    for i, lab in enumerate([r"$y$ [m]", r"$\dot y$ [m/s]", r"$\ddot y$ [m/s$^2$]"]):
        ax.plot(t, X[:, i], lw=0.7, label=lab)
    ax.set_title(f"Off-policy data collection (first 6 s of the {t[-1]:.0f} s record)")
    ax.set_xlabel("t [s]"); ax.set_ylabel("state (per-signal units)")
    ax.legend(ncol=3); ax.grid(alpha=.3)
    ax.set_xlim(0, min(6, t[-1]))
    fig.tight_layout(); fig.savefig(f"{OUT}/data.png", dpi=140); plt.close()

    # policy-iteration convergence: P eigenvalues, residual, coefficients, cost
    Peig = np.array(hist["P_eig"]); res = np.array(hist["res"])
    Wev = np.array(hist["w"])                          # (iterations, 12)
    it = np.arange(1, len(Wev) + 1)
    Jit = {}                                           # J_T of each intermediate policy
    for j in (0, 3):                                   # IC1 and IC4
        Jit[j] = [rollout(ics[j], np.asarray(wi), use_policy=True)[3] for wi in hist["w"]]
    fig, ax = plt.subplots(1, 4, figsize=(17, 3.8))
    for i in range(3):
        ax[0].plot(it, Peig[:, i], "o-", lw=1.4, label=f"$\\lambda_{i+1}(P)$")
    ax[0].axhline(0, color="k", lw=.7)
    ax[0].set_title("Eigenvalues of $P_i$ (stay $>0$)")
    ax[0].set_xlabel("iteration"); ax[0].set_ylabel("eig($P_i$)"); ax[0].legend(); ax[0].grid(alpha=.3)
    ax[1].semilogy(it, res + 1e-18, "s-", color="purple", lw=1.4)
    ax[1].set_title("Off-policy least-squares residual")
    ax[1].set_xlabel("iteration"); ax[1].set_ylabel("residual RMS"); ax[1].grid(alpha=.3, which="both")
    for i in range(Wev.shape[1]):
        ax[2].plot(it, Wev[:, i], "o-", lw=1.0, ms=3)
    ax[2].set_title(f"Value-function coefficients $w_i$ ({Wev.shape[1]} traces)")
    ax[2].set_xlabel("iteration"); ax[2].set_ylabel("$w_i$"); ax[2].grid(alpha=.3)
```

```
    ax[3].plot(it, Jit[0], "o-", color="tab:blue", lw=1.4, label="IC1")
    ax[3].plot(it, Jit[3], "s-", color="tab:red", lw=1.4, label="IC4")
    ax[3].axhline(J0[0], ls="--", color="tab:blue", alpha=.5, label=r"IC1: $J_T(u_0)$")
    ax[3].axhline(J0[3], ls="--", color="tab:red", alpha=.5, label=r"IC4: $J_T(u_0)$")
    ax[3].set_title("Cost $J_T$ of the intermediate policies")
    ax[3].set_xlabel("iteration"); ax[3].set_ylabel("$J_T$")
    ax[3].legend(fontsize=9); ax[3].grid(alpha=.3)
    fig.tight_layout(); fig.savefig(f"{OUT}/convergence.png", dpi=140); plt.close()
    print("J_T per iteration (IC1):", np.round(Jit[0], 2))

    # B7: learned vs initial policy from several ICs -- ALL states + input
    sel = [0, 1, 3]                                  # IC1, IC2, IC4 (IC4 = the Part-4 demo
     IC)
    cols = ["tab:blue", "tab:green", "tab:red"]
    fig, ax = plt.subplots(4, 1, figsize=(10, 10.5), sharex=True)
    labs = [r"$x_1=y$ [m]", r"$x_2=\dot y$ [m/s]", r"$x_3=\ddot y$ [m/s$^2$]"]
    for c, j in zip(cols, sel):
        tl, Xl, Ul, _ = rollout(ics[j], w, use_policy=True)
        t0, X0r, _, _ = rollout(ics[j], w, use_policy=False)
        for i in range(3):
            ax[i].plot(tl, Xl[:, i], color=c, lw=1.3,
                       label=f"IC{j+1} learned" if i == 0 else None)
            ax[i].plot(t0, X0r[:, i], color=c, lw=1.0, ls=":", alpha=.75,
                       label=f"IC{j+1} initial $u_0=0$" if i == 0 else None)
        ax[3].plot(tl, Ul, color=c, lw=1.2, label=f"IC{j+1}")
    for i in range(3):
        ax[i].set_ylabel(labs[i]); ax[i].grid(alpha=.3)
    ax[0].legend(ncol=3, fontsize=9)
    ax[3].set_ylabel("u [V]"); ax[3].set_xlabel("t [s]"); ax[3].grid(alpha=.3)
    ax[3].legend(ncol=3, fontsize=9, title="learned policy")
    fig.suptitle("Learned vs initial policy: all state variables and the control input")
    fig.tight_layout(); fig.savefig(f"{OUT}/regulation.png", dpi=140); plt.close()

    # cost comparison bar
    fig, ax = plt.subplots(figsize=(8, 4))
    xpos = np.arange(len(ics))
    ax.bar(xpos - 0.2, J0, 0.4, label=r"initial $u_0=0$", color="salmon")
    ax.bar(xpos + 0.2, Jl, 0.4, label="learned policy", color="steelblue")
    ax.set_xticks(xpos); ax.set_xticklabels([f"IC{i+1}" for i in range(len(ics))])
    ax.set_title(r"Accumulated cost $J_T$: learned policy vs initial policy")
    ax.set_ylabel(r"$J_T$"); ax.legend(); ax.grid(alpha=.3, axis="y")
    fig.tight_layout(); fig.savefig(f"{OUT}/cost.png", dpi=140); plt.close()
    print(f"\nsaved figures -> {OUT}/")

    # export identified coefficients for the Pico (Part 4)
    np.savez("data/adp_policy.npz", w=w, MON=MON, Q=np.diag(Q), R=R, c=P.c)
    print("saved data/adp_policy.npz")
```

A.4 Pico – process simulation on the secondary core `plant_core1.py`

```
"""
Secondary-core (core 1) task: real-time simulation of the continuous nonlinear
process.  One RK4 integration step is performed per paced-loop step (period dt),
with the controller's input u held constant between controller updates (ZOH).

f(x) is evaluated HERE only -- it never appears in the controller code, honouring
the "f unknown to the controller, simulated inside the process" requirement (C1).

Runs unchanged on the Pico (ulab) and on a desktop (NumPy fallback) for validation.
The RK4 step is written in scalar arithmetic (no array temporaries): on the RP2040
(Cortex-M0+, software floats) small-vector ulab calls are dominated by per-call
overhead, and the scalar form brings the step from ~2.9 ms to well under the plant
period; @micropython.native compiles it with the native emitter on the board.
"""
try:
    from ulab import numpy as np           # on the Pico
except ImportError:
    import numpy as np                     # on a desktop (validation)

try:
    import micropython                     # on the Pico: enables @micropython.native
```

```
except ImportError:
    class _shim:                            # desktop: identity decorator
        @staticmethod
        def native(fun):
            return fun
    micropython = _shim()

import params as PA


def f(x):
    """Drift f(x) of x' = f(x) + g u  (control-affine, g=[0,0,C_IN])."""
    x1, x2, x3 = x[0], x[1], x[2]
    D = PA.A1 * x3 + PA.A2 * x2 + PA.A3 * x1 + PA.B1 * x1 * x1 * x1 + PA.B2 * x1 * x1 *
    x2
    return np.array([x2, x3, -D])


def dyn(x, u):
    fx = f(x)
    return np.array([fx[0], fx[1], fx[2] + PA.C_IN * u])


@micropython.native
def rk4_scalars(x1, x2, x3, u, dt,
                A1=PA.A1, A2=PA.A2, A3=PA.A3, B1=PA.B1, B2=PA.B2, C_IN=PA.C_IN):
    """One RK4 step with ZOH input u held over dt, in scalar form (the core-1
    real-time loop).  The model constants are bound as default arguments so
    lookups stay local; no array temporaries are created on the plant core."""
    bu = C_IN * u
    h = 0.5 * dt

    k1a = x2; k1b = x3
    k1c = bu - (A1 * x3 + A2 * x2 + A3 * x1 + B1 * x1 * x1 * x1 + B2 * x1 * x1 * x2)

    e1 = x1 + h * k1a; e2 = x2 + h * k1b; e3 = x3 + h * k1c
    k2a = e2; k2b = e3
    k2c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)

    e1 = x1 + h * k2a; e2 = x2 + h * k2b; e3 = x3 + h * k2c
    k3a = e2; k3b = e3
    k3c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)

    e1 = x1 + dt * k3a; e2 = x2 + dt * k3b; e3 = x3 + dt * k3c
    k4a = e2; k4b = e3
    k4c = bu - (A1 * e3 + A2 * e2 + A3 * e1 + B1 * e1 * e1 * e1 + B2 * e1 * e1 * e2)

    w = dt / 6.0
    return (x1 + w * (k1a + 2.0 * (k2a + k3a) + k4a),
            x2 + w * (k1b + 2.0 * (k2b + k3b) + k4b),
            x3 + w * (k1c + 2.0 * (k2c + k3c) + k4c))


def rk4_step(x, u, dt):
    """Array-in / array-out wrapper around rk4_scalars (desktop validation and
    any caller holding the state as a vector)."""
    y1, y2, y3 = rk4_scalars(float(x[0]), float(x[1]), float(x[2]), u, dt)
    return np.array([y1, y2, y3])
```

A.5 Pico – controllers on the main core `pico_control.py`

```
"""
Main-core (core 0) controllers for the Pico: the adaptive RBF-NN tracking
controller (C3) and the frozen ADP optimal-regulation policy (C4).

All vector/matrix work uses ulab.numpy (the RBF feature vector and its inner
products are vectorised, not Python loops -- C5).  The same code runs on a desktop
via the NumPy fallback so the Pico results can be checked against the computer
results under matching conditions.
"""
try:
```

```
    from ulab import numpy as np           # on the Pico
    _ON_PICO = True
except ImportError:
    import numpy as np                     # on a desktop (validation)
    _ON_PICO = False

import math
import params as PA


def _sat(u, umax):
    return umax if u > umax else (-umax if u < -umax else u)


def y_ref(t):
    """(y_d, y_d', y_d'', y_d''') for y_d = A sin(w t)."""
    A, w = PA.REF_A, PA.REF_W
    s, c = math.sin(w * t), math.cos(w * t)
    return A * s, A * w * c, -A * w * w * s, -A * w * w * w * c


# ------------------------------------------------------------------------
# Branch A -- adaptive RBF-NN tracking controller
# ------------------------------------------------------------------------
class RBFController:
    def __init__(self):
        b = PA.RBF_BOX
        g = PA.RBF_GRID
        ax = [self._lin(b[i][0], b[i][1], g[i]) for i in range(3)]
        c0, c1, c2 = [], [], []
        for v0 in ax[0]:
            for v1 in ax[1]:
                for v2 in ax[2]:                # 'ij' ravel order (matches desktop)
                    c0.append(v0); c1.append(v1); c2.append(v2)
        self.C0 = np.array(c0); self.C1 = np.array(c1); self.C2 = np.array(c2)
        self.n = len(c0)
        self.ih0 = 2.0 / (b[0][1] - b[0][0])
        self.ih1 = 2.0 / (b[1][1] - b[1][0])
        self.ih2 = 2.0 / (b[2][1] - b[2][0])
        self.i2w2 = 1.0 / (2.0 * PA.RBF_WIDTH * PA.RBF_WIDTH)
        # pre-scaled PER-AXIS centers: d_i = x_i*m_i - A_i gives
        # (x_i-c_i)*ih_i*sqrt(i2w2) in one small (grid-length) vector op, so
        # exp(-(d0^2+d1^2+d2^2)) needs no further scaling
        s = math.sqrt(self.i2w2)
        self.m0 = self.ih0 * s; self.A0 = np.array(ax[0]) * self.m0
        self.m1 = self.ih1 * s; self.A1 = np.array(ax[1]) * self.m1
        self.m2 = self.ih2 * s; self.A2 = np.array(ax[2]) * self.m2
        self.g0, self.g1, self.g2 = g[0], g[1], g[2]
        self.W = np.zeros(self.n)
        self.l2, self.l1, self.l0 = PA.LAM
        self.bp0, self.bp1, self.bp2 = PA.BP

    @staticmethod
    def _lin(a, b, n):
        return [a] if n == 1 else [a + (b - a) * i / (n - 1) for i in range(n)]

    def _gauss(self, x):
        """Unnormalized Gaussian vector g(x), exploiting the SEPARABILITY of the
        grid: exp(-(d0^2+d1^2+d2^2)) over the 6x6x6 Cartesian product equals the
        outer product of three per-axis 6-element Gaussians.  All exp() calls act
        on grid-length vectors; only two broadcast multiplications touch the full
        216-element vector -- on the RP2040 @ 200 MHz this brings the full RBF
        update to the measured ~2.9 ms.  The (g0*g1)*g2 broadcast order reproduces the 'ij' ravel order
        of the flat center list, so W indexing is unchanged."""
        d0 = x[0] * self.m0 - self.A0
        d1 = x[1] * self.m1 - self.A1
        d2 = x[2] * self.m2 - self.A2
        d0 *= d0; d1 *= d1; d2 *= d2
        d0 *= -1.0; d1 *= -1.0; d2 *= -1.0
        e0 = np.exp(d0); e1 = np.exp(d1); e2 = np.exp(d2)
        a = e0.reshape((self.g0, 1)) * e1.reshape((1, self.g1))
        b = a.reshape((self.g0 * self.g1, 1)) * e2.reshape((1, self.g2))
```

```
        return b.reshape((self.n,))

    def features(self, x):
        """Normalized Gaussian RBF vector phi(x) (vectorised ulab op)."""
        g = self._gauss(x)
        s = np.sum(g)
        return g / s if s > 1e-12 else g   # underflow guard, same as the desktop

    def update(self, x, t, dt):
        """One controller update: read state x at time t, return u; adapt weights.
        dt is the controller sampling period Ts (forward-Euler weight update).
        The normalization phi = g/sum(g) is folded into the two scalar results
        (W.phi = W.g/sum(g); W-update scaled by 1/sum(g)), saving a 216-element
        vector division per update on the Pico."""
        yd, yd1, yd2, yd3 = y_ref(t)
        e = x[0] - yd; ed = x[1] - yd1; edd = x[2] - yd2
        g = self._gauss(x)
        s = np.sum(g)
        # underflow guard (matches the desktop rbf_phi): far outside the center
        # box the float32 sum can reach 0, and dividing would give NaN weights
        isg = 1.0 / s if s > 1e-12 else 0.0
        Dhat = np.dot(self.W, g) * isg
        u = _sat((Dhat + yd3 - self.l2 * edd - self.l1 * ed - self.l0 * e) / PA.C_IN, PA.
    U_MAX)
        # Lyapunov gradient weight update + projection (forward Euler, in place)
        bPxi = self.bp0 * e + self.bp1 * ed + self.bp2 * edd
        g *= dt * PA.GAMMA_ADAPT * bPxi * isg
        self.W -= g
        self.W = np.clip(self.W, -PA.W_MAX, PA.W_MAX)      # projection
        return u


# ---------------------------------------------------------------------------
# Branch B -- frozen ADP optimal-regulation policy  u = -(1/2R) zeta(x)^T w
# ---------------------------------------------------------------------------
class ADPController:
    def __init__(self):
        self.w = np.array(PA.ADP_W)
        self.mon = PA.ADP_MON
        self.k = -1.0 / (2.0 * PA.ADP_R)

    def zeta(self, x):
        """c * d/dx3 of the value-function basis [quadratic ; nonlinear]."""
        x1, x2, x3 = x[0], x[1], x[2]
        z = [0.0, 0.0, x1, 0.0, x2, 2.0 * x3]              # d/dx3 of quadratic monomials
        for e in self.mon:                                 # d/dx3 of nonlinear monomials
            if e[2] == 0:
                z.append(0.0)
            else:
                z.append(e[2] * x1 ** e[0] * x2 ** e[1] * x3 ** (e[2] - 1))
        return np.array(z) * PA.C_IN

    def control(self, x):
        return _sat(self.k * np.dot(self.zeta(x), self.w), PA.U_MAX)
```

A.6 Pico – two-core real-time orchestration `main.py`

```
"""
Raspberry Pi Pico top-level (Part 4).  Runs the complete two-core real-time scheme:

    core 1 (second core)  : simulates the nonlinear process.  A paced real-time
                            loop performs ONE scalar RK4 step every dt with the
                            controller input u held constant (ZOH) and catch-up if
                            a step is transiently delayed.  (A machine.Timer is
                            NOT used here: on the RP2040 port soft-timer callbacks
                            are always dispatched on core 0 -- verified on hardware
                            via the SIO CPUID register -- so a paced loop inside
                            the _thread task is what actually runs on core 1.)
    core 0 (main core)    : a hardware Timer with period Ts = N*dt triggers ONE
                            controller update -- it reads the latest full state,
                            computes u, (adapts the RBF weights,) and publishes u
```

```
                            to core 1.  The per-update execution time is measured
                            and its average and maximum are reported (must stay
                            below Ts).  gc.collect() runs every GC_EVERY ticks so
                            collection happens at a controlled point.

Set MODE = "rbf" (adaptive tracking, C3) or "adp" (frozen optimal policy, C4).

This module targets MicroPython on the Pico (machine, _thread, ulab).  It is not run
on the desktop; the numerical logic is validated off-board by desktop_sim.py, which
imports the same params/plant_core1/pico_control unchanged.
"""
import _thread
import gc
import utime
import machine
from machine import Timer

import params as PA
import plant_core1 as PL
import pico_control as CT

machine.freq(200_000_000)       # 200 MHz, the officially supported RP2040 fast clock

MODE = "rbf"                    # "rbf" (Branch A tracking) or "adp" (Branch B regulation
    )
X0 = [0.0, 0.0, 0.0] if MODE == "rbf" else [0.7, 0.5, -1.0]
GC_EVERY = 10                   # controller ticks between deterministic collections

# ---- shared state between the cores (single-writer each; lock guards exchange) ----
_s1, _s2, _s3 = X0              # state scalars, written by core 1, read by core 0
_u = 0.0                        # written by core 0, read by core 1
_lock = _thread.allocate_lock()
_go = False                     # plant holds X0 until the first u is published

# ---- execution-time statistics (C5) ----
_exec_sum = 0
_exec_max = 0
_exec_cnt = 0


# ------------------------------------------------------------------------------
# core 1: process simulation, one scalar RK4 step every dt (paced loop)
# ------------------------------------------------------------------------------
def core1_task():
    global _s1, _s2, _s3
    dt = PA.PICO_DT
    dt_us = int(dt * 1e6)
    x1, x2, x3 = _s1, _s2, _s3
    while not _go:                      # aligned start: wait for u(X0)
        pass
    nxt = utime.ticks_add(utime.ticks_us(), dt_us)
    while True:
        if utime.ticks_diff(utime.ticks_us(), nxt) < 0:
            continue                    # busy-wait: the core is dedicated
        nxt = utime.ticks_add(nxt, dt_us)
        with _lock:
            u = _u
        x1, x2, x3 = PL.rk4_scalars(x1, x2, x3, u, dt)
        with _lock:
            _s1 = x1; _s2 = x2; _s3 = x3


# ------------------------------------------------------------------------------
# core 0: controller update, one per timer tick (period Ts = N*dt)
# ------------------------------------------------------------------------------
def make_controller_tick(ctrl):
    box = {"k": 0}

    def _tick(timer):
        global _u, _go, _exec_sum, _exec_max, _exec_cnt
        k = box["k"]
        ta = utime.ticks_us()
        with _lock:
```

```
            x = (_s1, _s2, _s3)
        if MODE == "rbf":
            u = ctrl.update(x, k * PA.PICO_TS, PA.PICO_TS)   # logical time: no
    wraparound
        else:
            u = ctrl.control(x)
        with _lock:
            _u = u
        dt_us = utime.ticks_diff(utime.ticks_us(), ta)   # read + compute + publish (C5)
        _go = True
        if k % GC_EVERY == 0:
            gc.collect()      # deterministic collection, never mid-update
        box["k"] = k + 1
        _exec_sum += dt_us
        _exec_cnt += 1
        if dt_us > _exec_max:
            _exec_max = dt_us
    return _tick


def main():
    ctrl = CT.RBFController() if MODE == "rbf" else CT.ADPController()
    _thread.start_new_thread(core1_task, ())        # plant on core 1
    utime.sleep_ms(50)                              # let core 1 start
    Timer(period=int(PA.PICO_TS * 1000), mode=Timer.PERIODIC,
          callback=make_controller_tick(ctrl))      # controller on core 0
    # report loop.  NOTE: no lock here -- the controller callback runs on THIS core
    # at bytecode boundaries, so taking _lock in this loop could deadlock against a
    # callback dispatched while the lock is held; a single stale float read is
    # perfectly fine for a 1 Hz diagnostic print.
    while True:
        utime.sleep_ms(1000)
        y = _s1
        avg = (_exec_sum / _exec_cnt) if _exec_cnt else 0
        print("MODE=%s  y=%.4f  u=%.3f  exec avg=%.1f us  max=%d us  (Ts=%d us)"
              % (MODE, y, _u, avg, _exec_max, int(PA.PICO_TS * 1e6)))


if __name__ == "__main__":
    main()
```

A.7 Pico – logged hardware experiment `pico_experiment.py`

```
"""
Bounded, logged run of the two-core real-time scheme (Part 4 hardware experiment).

Core allocation (same intent as main.py):

    core 1 : the nonlinear process.  A paced real-time loop performs one scalar
             RK4 step (plant_core1.rk4_scalars) every dt, with catch-up if a step
             is transiently delayed (e.g. by a GC pause on the other core).  A
             plain machine.Timer is NOT used for the plant: on the RP2040 port
             MicroPython soft-timer callbacks are always dispatched on core 0
             (verified on hardware via the SIO CPUID register), so a paced loop
             inside the _thread task is the only way to actually run the process
             on the second core.
    core 0 : the controller, triggered by a hardware Timer every Ts = N*dt.  It
             copies the three state scalars under the lock, computes u, publishes
             it, and logs (y, u, exec time).  gc.collect() runs every GC_EVERY
             ticks so collection happens at a controlled point, never mid-update.

The run lasts a fixed simulated time T; the log is printed as CSV over USB so the
desktop side (plot_hw_results.py) can draw the Pico-vs-computer overlays and the
timing histogram from real on-chip data.

Usage from the host:

    mpremote exec "import pico_experiment; pico_experiment.run('rbf', 30.0)"
    mpremote exec "import pico_experiment; pico_experiment.run('adp',  8.0)"

Each call shuts the timer down and releases the second core, so both runs can be
```

```
captured in one session.
"""
import _thread
import array
import gc
import machine
import utime
from machine import Timer

import params as PA
import plant_core1 as PL
import pico_control as CT

machine.freq(200_000_000)      # officially supported RP2040 fast clock

_CPUID = 0xD0000000            # SIO CPUID register: 0 on core 0, 1 on core 1
GC_EVERY = 10                  # controller ticks between deterministic collections

# shared between the cores (single-writer each; lock guards the exchange)
_s1 = _s2 = _s3 = 0.0          # state scalars, written by core 1
_u = 0.0                       # input, written by core 0
_lock = _thread.allocate_lock()
_plant_ticks = 0
_plant_cpu = -1
_plant_late_max = 0            # worst plant-step start delay [us]
_core1_stop = False
_core1_done = False
_go = False                    # plant holds x0 until the first u is published


def _core1_task():
    """Plant real-time loop on the second core: one RK4 step every dt."""
    global _s1, _s2, _s3, _plant_ticks, _plant_cpu, _core1_done, _plant_late_max
    _plant_cpu = machine.mem32[_CPUID] & 1
    dt = PA.PICO_DT
    dt_us = int(dt * 1e6)
    x1, x2, x3 = _s1, _s2, _s3
    while not _go and not _core1_stop:      # aligned start: wait for u(x0)
        pass
    nxt = utime.ticks_add(utime.ticks_us(), dt_us)
    while not _core1_stop:
        late = utime.ticks_diff(utime.ticks_us(), nxt)
        if late < 0:
            continue                        # busy-wait: the core is dedicated
        if late > _plant_late_max:
            _plant_late_max = late
        nxt = utime.ticks_add(nxt, dt_us)
        with _lock:
            u = _u
        x1, x2, x3 = PL.rk4_scalars(x1, x2, x3, u, dt)
        with _lock:
            _s1 = x1; _s2 = x2; _s3 = x3
        _plant_ticks += 1
    _core1_done = True


def run(mode="rbf", T=20.0):
    global _s1, _s2, _s3, _u, _core1_stop, _core1_done, _plant_ticks, _plant_late_max,
    _go
    K = int(round(T / PA.PICO_TS))
    Ts = PA.PICO_TS

    y_log = array.array("f", bytearray(4 * K))
    u_log = array.array("f", bytearray(4 * K))
    exec_log = array.array("H", bytearray(2 * K))    # controller update [us]
    tot_log = array.array("H", bytearray(2 * K))     # full tick incl. log/gc [us]

    ctrl = CT.RBFController() if mode == "rbf" else CT.ADPController()
    _s1, _s2, _s3 = (0.0, 0.0, 0.0) if mode == "rbf" else (0.7, 0.5, -1.0)
    _u = 0.0
    _plant_ticks = 0
    _plant_late_max = 0
    _core1_stop = False
```

```
    _core1_done = False
    _go = False

    box = {"k": 0, "cpu": -1}

    def _ctrl_tick(timer):
        global _u, _go
        k = box["k"]
        if k >= K:
            return
        box["cpu"] = machine.mem32[_CPUID] & 1
        ta = utime.ticks_us()
        with _lock:
            x = (_s1, _s2, _s3)
        if mode == "rbf":
            u = ctrl.update(x, k * Ts, Ts)
        else:
            u = ctrl.control(x)
        with _lock:
            _u = u
        te = utime.ticks_diff(utime.ticks_us(), ta)   # read + compute + publish (C5)
        _go = True                  # release the plant on the first published u
        y_log[k] = x[0]
        u_log[k] = u
        if k % GC_EVERY == 0:
            gc.collect()      # collection at a controlled point, never mid-update
        tt = utime.ticks_diff(utime.ticks_us(), ta)
        exec_log[k] = te if te < 65535 else 65535
        tot_log[k] = tt if tt < 65535 else 65535
        box["k"] = k + 1

    gc.collect()
    _thread.start_new_thread(_core1_task, ())
    utime.sleep_ms(100)                         # let the plant loop start
    tim0 = Timer(period=int(Ts * 1000), mode=Timer.PERIODIC, callback=_ctrl_tick)

    t_wall = utime.ticks_ms()
    while box["k"] < K:
        utime.sleep_ms(50)
    wall = utime.ticks_diff(utime.ticks_ms(), t_wall)

    tim0.deinit()
    _core1_stop = True
    while not _core1_done:
        utime.sleep_ms(5)

    ex = exec_log[1:]                           # skip the cold first tick
    print("# mode=%s T=%.1f Ts=%g dt=%g K=%d plant_ticks=%d wall_ms=%d plant_late_max_us
    =%d" %
          (mode, T, Ts, PA.PICO_DT, K, _plant_ticks, wall, _plant_late_max))
    print("# freq_hz=%d ctrl_cpu=%d plant_cpu=%d micropython=%s" %
          (machine.freq(), box["cpu"], _plant_cpu,
           ".".join(str(v) for v in __import__("sys").implementation.version[:3])))
    print("# exec_us avg=%.1f max=%d | tick_us(incl log/gc) avg=%.1f max=%d | Ts_us=%d" %
          (sum(ex) / len(ex), max(ex), sum(tot_log[1:]) / (K - 1), max(tot_log[1:]),
           int(Ts * 1e6)))
    print("k,y,u,exec_us,tot_us")
    for k in range(K):
        print("%d,%.6f,%.5f,%d,%d" % (k, y_log[k], u_log[k], exec_log[k], tot_log[k]))
    print("# done")
```